



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Specifica Tecnica

Stato	In lavorazione
Versione	0.6.0
Autori	Aldo Bettega Davide Lorenzon Ana Maria Draghici Filippo Guerra Felician Mario Necsulescu
Verificatori	Davide Lorenzon Filippo Guerra Felician Mario Necsulescu
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin BlueWind srl

Vers.	Data	Autore	Verificatore	Descrizione
0.7.1	2026-04-20	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern comportamentali e architetturali <u>Sezione 4.5, Sezione 4.2</u>
0.7.1	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern: creazionali e strutturali <u>Sezione 4.3, Sezione 4.4</u>
0.7.0	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei principi di design <u>Sezione 4.1</u>
0.6.4	2026-04-19	Davide Lorenzon	-	Revisione architetturale della sezione relativa alla modifica degli asset <u>Sezione 5.3</u>
0.6.3	2026-04-19	Davide Lorenzon	-	Bozza delle classi relative alla generazione del report di conformità <u>Sezione 5.7</u>
0.6.2	2026-04-19	Davide Lorenzon	-	Bozza delle classi relative a import ed export tramite file di dispositivo e modello <u>Sezione 5.5 e Sezione 5.6</u>
0.6.1	2026-04-17	Filippo Guerra	-	Bozza della classe valutazione <u>Sezione 5.4</u>
0.6.0	2026-04-17	Filippo Guerra	Davide Lorenzon	Stesura vista_dati <u>Sezione 5.3</u>
0.5.0	2026-04-16	Ana Maria Draghici	Davide Lorenzon	Stesura vista_dati <u>Sezione 3.6</u>
0.4.0	2026-04-16	Aldo Bettega	Davide Lorenzon	Stesura <u>Sezione 5.1 e Sezione 5.2</u>
0.3.0	2026-04-15	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.4</u>
0.2.0	2026-04-10	Aldo Bettega	Davide Lorenzon	Stesura della <u>Sezione 3.1 e Sezione 3.2</u>
0.1.2	2026-04-09	Davide Lorenzon	Filippo Guerra	Bozza iniziale della <u>Sezione 3.3</u> Architettura di deployment.

Vers.	Data	Autore	Verificatore	Descrizione
0.1.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Bozza iniziale della <u>Sezione 2</u> Tecnologie.
0.1.0	2026-04-08	Aldo Bettega	Felician Mario Necsulescu	Stesura introduzione
0.0.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Creazione del documento

Indice

1) Introduzione	1
1.1) Scopo del documento	1
1.2) Scopo del prodotto	1
1.3) Glossario	1
1.4) Riferimenti	2
1.4.1) Riferimenti normativi	2
1.4.2) Riferimenti informativi	2
2) Tecnologie	3
2.1) Backend	3
2.2) Frontend	5
2.3) Persistenza dei dati	6
3) Architettura di Sistema	7
3.1) Premessa: approccio di lavoro usato	7
3.2) Architettura Logica	7
3.2.1) Stile architetturale	7
3.2.2) Motivazioni della scelta architetturale	8
3.2.3) Limiti dell'architettura	8
3.2.4) Diagramma dei package	9
3.3) Architettura di Deployment	11
3.3.1) Motivazioni della scelta	11
3.3.2) Realizzazione Pratica e Topologia	11
3.4) Diagrammi di sequenza	12
3.4.1) UC05 - Importazione dispositivo	12
3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato	13
3.4.3) UC30 - Esportazione report di conformità	14
3.5) Diagrammi di attività	15
3.5.1) Navigazione degli alberi	15
3.6) Schema dati	16
3.6.1) Scelte Architetturali e Formalismo	16
3.6.2) Schema Dati Dispositivo	17
3.6.3) Schema Dati Modello	19
3.6.4) Gestione degli Esiti e Storicità	22
4) Design Patterns	23
4.1) Principi di Design	23
4.1.1) Dependency Injection	23
4.1.1.1) Descrizione del pattern	23
4.1.1.2) Motivazioni dell'utilizzo del pattern	23
4.1.1.3) Utilizzo del pattern nel progetto	23
4.2) Pattern Architetturali	23
4.2.1) Repository	23

4.2.1.1)	Descrizione del pattern	23
4.2.1.2)	Motivazioni dell'utilizzo del pattern	23
4.2.1.3)	Utilizzo del pattern nel progetto	23
4.2.2)	Unit of Work	24
4.2.2.1)	Descrizione del pattern	24
4.2.2.2)	Motivazioni dell'utilizzo del pattern	24
4.2.2.3)	Utilizzo del pattern nel progetto	24
4.3)	Design Patterns Creazionali	24
4.3.1)	Factory	24
4.3.1.1)	Descrizione del pattern	24
4.3.1.2)	Motivazioni dell'utilizzo del pattern	24
4.3.1.3)	Utilizzo del pattern nel progetto	25
4.4)	Design Patterns strutturali	25
4.4.1)	Adapter	25
4.4.1.1)	Descrizione del pattern	25
4.4.1.2)	Motivazioni dell'utilizzo del pattern	25
4.4.1.3)	Utilizzo del pattern nel progetto	25
4.4.2)	Facade	25
4.4.2.1)	Descrizione del pattern	25
4.4.2.2)	Motivazioni dell'utilizzo del pattern	26
4.4.2.3)	Utilizzo del pattern nel progetto	26
4.5)	Design Patterns Comportamentali	26
4.5.1)	Strategy	26
4.5.1.1)	Descrizione del pattern	26
4.5.1.2)	Motivazioni dell'utilizzo del pattern	26
4.5.1.3)	Utilizzo del pattern nel progetto	26
4.5.2)	Template Method	26
4.5.2.1)	Descrizione del pattern	26
4.5.2.2)	Motivazioni dell'utilizzo del pattern	27
4.5.2.3)	Utilizzo del pattern nel progetto	27
5)	Diagrammi delle classi	28
5.1)	Diagramma di dominio	28
5.2)	Dispositivo	30
5.3)	Asset	32
5.4)	Valutazione	34
5.5)	Importazione ed Esportazione Dispositivi	36
5.5.1)	Importazione Dispositivo	37
5.5.2)	Esportazione dispositivo	38
5.5.3)	Classi Condivise	39
5.6)	Importazione ed Esportazione Modelli	39
5.6.1)	Importazione Modello	39

5.6.2) Esportazione modello	40
5.6.3) Classi Condivise	41
5.7) Generazione Report	41
6) Tracciamento	43
7) Qualità architetture	44
8) Gestione Errori e Logging	45
9) Sicurezza	46
10) Performance e Scalabilità	47

Lista delle tabelle

Tabella 1	Backend	3
Tabella 2	Frontend	5
Tabella 3	Schema dati: Root — Dispositivo	18
Tabella 4	Schema dati: Sub-documento — ref_modello	18
Tabella 5	Schema dati: Sub-documento — lista_asset[N]	18
Tabella 6	Schema dati: Sub-documento — lista_valutazioni[N]	19
Tabella 7	Schema dati: Root — Modello	21
Tabella 8	Schema dati: Sub-documento — lista_requisiti[N]	21
Tabella 9	Schema dati: Sub-documento — Nodo_decisione	21
Tabella 10	Schema dati: Sub-documento — Nodo_foglia	22

Lista delle immagini

Figura 1	Schema logico: Documento Dispositivo	17
Figura 2	Schema logico: Documento Modello	20
Figura 3	Diagramma delle classi - Importazione Dispositivo	37
Figura 4	Diagramma delle classi - Esportazione Dispositivo	38
Figura 5	Diagramma delle classi - Importazione Modello	39
Figura 6	Diagramma delle classi - Esportazione Modello	40
Figura 7	Diagramma delle classi - Importazione Modello	41

1) Introduzione

1.1) Scopo del documento

Il presente documento intende fornire una descrizione approfondita dell'architettura del prodotto software, delineando con chiarezza le componenti che lo costituiscono, le loro responsabilità e le interazioni reciproche all'interno del sistema. La Specifica Tecnica funge da riferimento principale per la fase di progettazione e codifica, garantendo coerenza con i requisiti analizzati e consolidando la maturità architetturale del prodotto.

Nello specifico, questo documento si propone di:

- **Definire l'architettura logica e i design pattern:** descrivere l'organizzazione dei moduli e le logiche di interazione, evidenziando come i pattern adottati garantiscano un codice modulare e testabile.
- **Motivare lo stack tecnologico:** giustificare la scelta delle tecnologie utilizzate in funzione della scalabilità del sistema e della qualità complessiva del prodotto finale.
- **Delineare l'architettura di deployment:** illustrare la distribuzione delle componenti negli ambienti di esecuzione e le strategie di rilascio adottate.
- **Fornire un framework per l'evoluzione del software :** stabilire le linee guida necessarie per facilitare la comprensione del sistema, agevolando futuri interventi di estensione e manutenzione.

1.2) Scopo del prodotto

Il prodotto si prefigge di automatizzare e digitalizzare il processo di verifica della conformità alla normativa EN 18031, sostituendo le attuali procedure manuali, spesso onerose e soggette a errore umano, con una soluzione software interattiva.

Il sistema è progettato per guidare l'utente attraverso l'esecuzione di alberi decisionali (Decision Tree) complessi, permettendo la valutazione sistematica dei requisiti di sicurezza per dispositivi IoT e la generazione di esiti certi (Pass, Fail, N.A.). L'integrazione di funzionalità per l'importazione di dati strutturati e una dashboard di monitoraggio in tempo reale mira a garantire la massima tracciabilità del processo, ottimizzando i tempi operativi della proponente e assicurando un elevato standard di affidabilità e manutenibilità dei risultati ottenuti.

1.3) Glossario

Per garantire precisione terminologica senza appesantire la lettura, in questo documento i termini tecnici presenti nel Glossario sono segnalati con un pedice "G", ad esempio:

termine_G: indica che il termine è definito nel Glossario e può essere consultato per chiarimenti.

Questo metodo consente di mantenere il testo chiaro e tecnicamente corretto, permettendo al lettore di riferirsi al Glossario solo quando necessario, senza interrompere il flusso della lettura.

1.4) Riferimenti

1.4.1) Riferimenti normativi

- [Capitolato d'appalto C1 - Automated EN18031 Compliance Verification di BlueWind Srl](#)
Ultima consultazione: 8 aprile 2026;
- [Regolamento del progetto](#)
Ultima consultazione: 8 aprile 2026;
- [Standard ISO/IEC/IEEE 12207:2017](#)
Ultima consultazione: 10 gennaio 2026;
- [Standard ISO/IEC 9126](#)
Ultima consultazione: 10 gennaio 2026;

1.4.2) Riferimenti informativi

- [Glossario del gruppo](#)
Ultima consultazione: 8 aprile 2026;
- [Software Engineering, Ian Sommerville](#)
Ultima consultazione: 10 gennaio 2026;
- [Documentazione del gruppo GroupRubberDuck](#)
Ultima consultazione: 8 aprile 2026;

2) Tecnologie

In questa sezione vengono descritte le tecnologie utilizzate nello sviluppo del progetto **Automated EN18031 Compliance Verification**.

Per ogni tecnologia è riportata la versione di riferimento e il relativo ruolo all'interno del progetto

2.1) Backend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
Python	3.12.X	Linguaggio interpretato scelto per la rapidità di sviluppo, l'alta leggibilità e il vasto ecosistema.
Framework Principale		
Flask	3.1.3	Micro-framework web utilizzato come infrastruttura principale di backend. Consente una gestione flessibile e leggera del routing per esporre le interfacce API. Buona documentazione e possibilità di confronto tecnico con la proponente.
Dipendenze principali		
waitress	3.0.2	Server WSGI leggero. Gestisce la concorrenza delle richieste in modo affidabile.
fpdf		Libreria per la generazione dinamica di documenti e reportistica in formato PDF direttamente lato server.
pymongo		Driver ufficiale per l'integrazione con MongoDB.
pydantic		Utilizzato per la validazione rigida e type-safe dei dati in ingresso e delle variabili d'ambiente di sistema.
python-dotenv		Gestione semplificata delle configurazioni e delle variabili d'ambiente.
Jinja2		Template engine HTML con un'ottima integrazione con Flask.

Nome	Versione	Descrizione
werkzeug		Server di sviluppo locale.
watchdog		Permette l'hot reloading, passando le modifiche al server di sviluppo senza necessità di riavvio e preservando lo stato dell'applicazione.
Dynamic Testing		
pytest		Framework di testing adottato per la sua sintassi concisa. Utilizzato per automatizzare i «Sanity Tests» e validare l'integrazione di sistema alla radice del progetto.
Static Testing		
ruff		Lint e formatter estremamente veloce (scritto in Rust). Impone e garantisce standard di codice puliti e uniformi senza rallentare lo sviluppo.
mypy		Analizzatore statico basato sul Type Hinting. Previene i bug e gli errori di tipo a tempo di sviluppo prima ancora dell'esecuzione
Sviluppo		
poetry		Gestore moderno delle dipendenze e degli ambienti virtuali (.venv). Garantisce build riproducibili tra i vari sviluppatori tramite il file di blocco (poetry.lock).

Tabella 1: Backend

2.2) Frontend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
JavaScript		Linguaggio client-side universale.
Framework Principale		
Vue.js		Framework progressivo con una curva di apprendimento morbida e un'ottima implementazione della reattività. L'adozione della Composition API permette di scrivere logica modulare e riutilizzabile.
Dipendenze principali		
pinia		Gestore dello stato ufficiale e leggero per Vue.
axios		Client HTTP flessibile. Preferito alla Fetch API nativa per l'ergonomia nella gestione automatica del JSON, la gestione degli errori e l'uso degli interceptor per le chiamate verso le API di Flask.
tailwindcss		Framework CSS utility-first per un'agile stilizzazione dell'interfaccia.
Dynamic Testing		
vitest		Framework di unit testing veloce e nativo per Vite. Utilizzato per validare la logica isolata dei componenti.
vue-test-utils		Libreria ufficiale affiancata a Vitest per montare e simulare le interazioni dell'utente sulle singole isole Vue durante i test.
Static Testing		
eslint		Linter per garantire standard di pulizia e coerenza nel codice JavaScript dei vari componenti Vue.
Sviluppo		

Nome	Versione	Descrizione
vite		Tool di build per minificare e raggruppare i file JS/CSS corrispondenti ai componenti Vue, ottimizzandoli per la produzione.

Tabella 2: Frontend

2.3) Persistenza dei dati

MongoDB è un database NoSQL orientato ai documenti. A differenza dei classici database relazionali (che usano tabelle e colonne rigide), archivia le informazioni in documenti flessibili molto simili al formato JSON.

La versione utilizzata è la 7.XX

Permette di aggiungere o rimuovere campi dai dati senza riscrivere l'intera organizzazione delle informazioni del database.

Nel progetto **Automated EN18031 Compliance Verification** viene utilizzato come sistema per la persistenza dei dati, in particolare per la rappresentazione dei dispositivi sottoposti alle valutazioni e per la rappresentazione del modello di standard tramite configurazioni esterne.

Il suo utilizzo permette di rappresentare facilmente strutture dati annidate, tramite una sintassi JSON.

Evita la gestione di join complessi tipici di un database relazionale, offre una maggiore sicurezza rispetto alla gestione manuale di file di rappresentazione interni.

È facilmente integrabile nel sistema backend

3) Architettura di Sistema

3.1) Premessa: approccio di lavoro usato

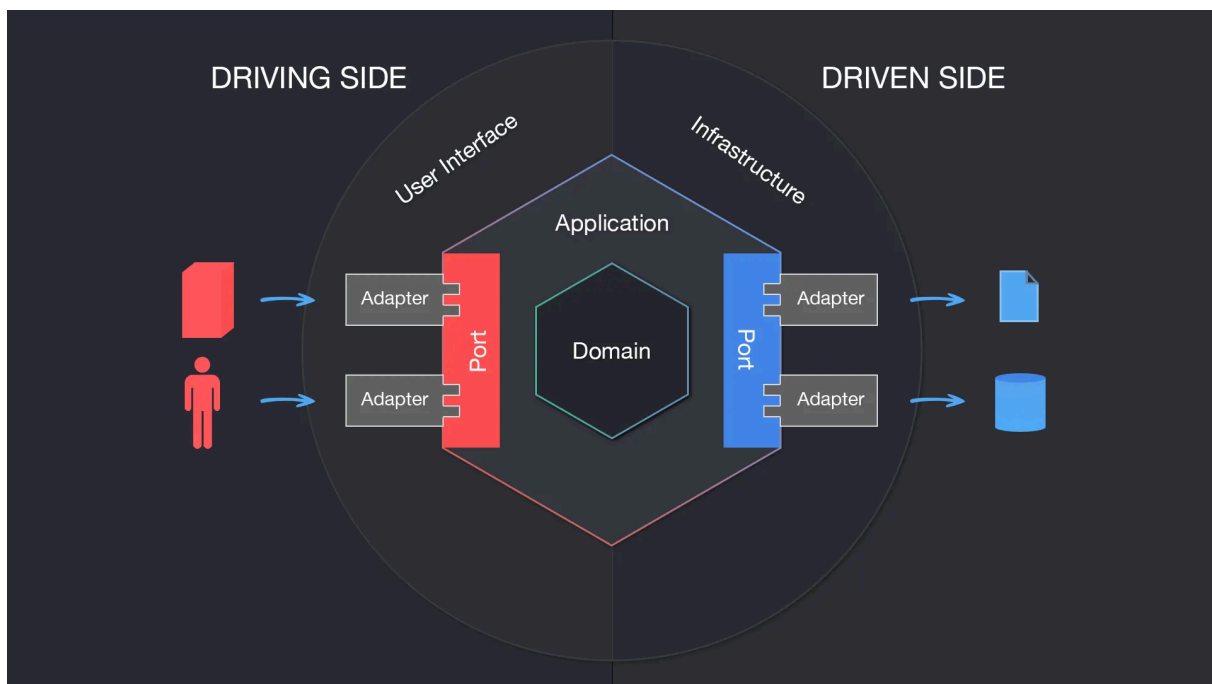
L'obiettivo della fase di progettazione è definire un'architettura software in grado di soddisfare pienamente i requisiti di sistema, operando secondo criteri di sostenibilità e ottimizzazione delle risorse (design-to-cost).

La metodologia adottata per la definizione dell'architettura è di tipo Top-Down. Il sistema viene inizialmente concepito nella sua interezza come un'unica entità astratta e, attraverso un processo di esplorazione funzionale, viene progressivamente decomposto in sotto-sistemi logici più piccoli e gestibili.

3.2) Architettura Logica

3.2.1) Stile architetturale

L'architettura adottata per realizzare il prodotto è l'architettura esagonale. Questo tipo di architettura ha come obiettivo primario isolare la logica di business dal resto. Questo approccio garantisce un'elevata testabilità del sistema e rende il nucleo del software completamente agnostico rispetto ai dettagli implementativi (framework, database o interfacce). Il sistema è strutturato in livelli concentrici, organizzati come segue:



- **Domain (Core):** Rappresenta il nucleo centrale dell'applicazione. Contiene la logica di business pura ed è rigorosamente privo di dipendenze esterne.
- **Ports:** Costituiscono il punto di connessione tra il nucleo e il mondo esterno, permettendo una comunicazione strutturata senza creare accoppiamento. Si suddividono in Inbound Ports (definiscono i casi d'uso accessibili dall'esterno) e Outbound Ports (permettono al nucleo di definire interfacce per interagire con i servizi esterni).

- **Adapters:** Rappresentano lo strato più esterno e fungono da traduttori tra le tecnologie specifiche e il nucleo. Si suddividono in Driving/Input Adapters (adattatori in entrata, che guidano l'applicazione ricevendo input e invocando le Inbound Ports) e Driven/Output Adapters (adattatori in uscita, che vengono guidati dall'applicazione per interagire con l'infrastruttura esterna tramite le Outbound Ports).

3.2.2) Motivazioni della scelta architetturale

Le motivazioni per le quali questa architettura è stata scelta sono le seguenti:

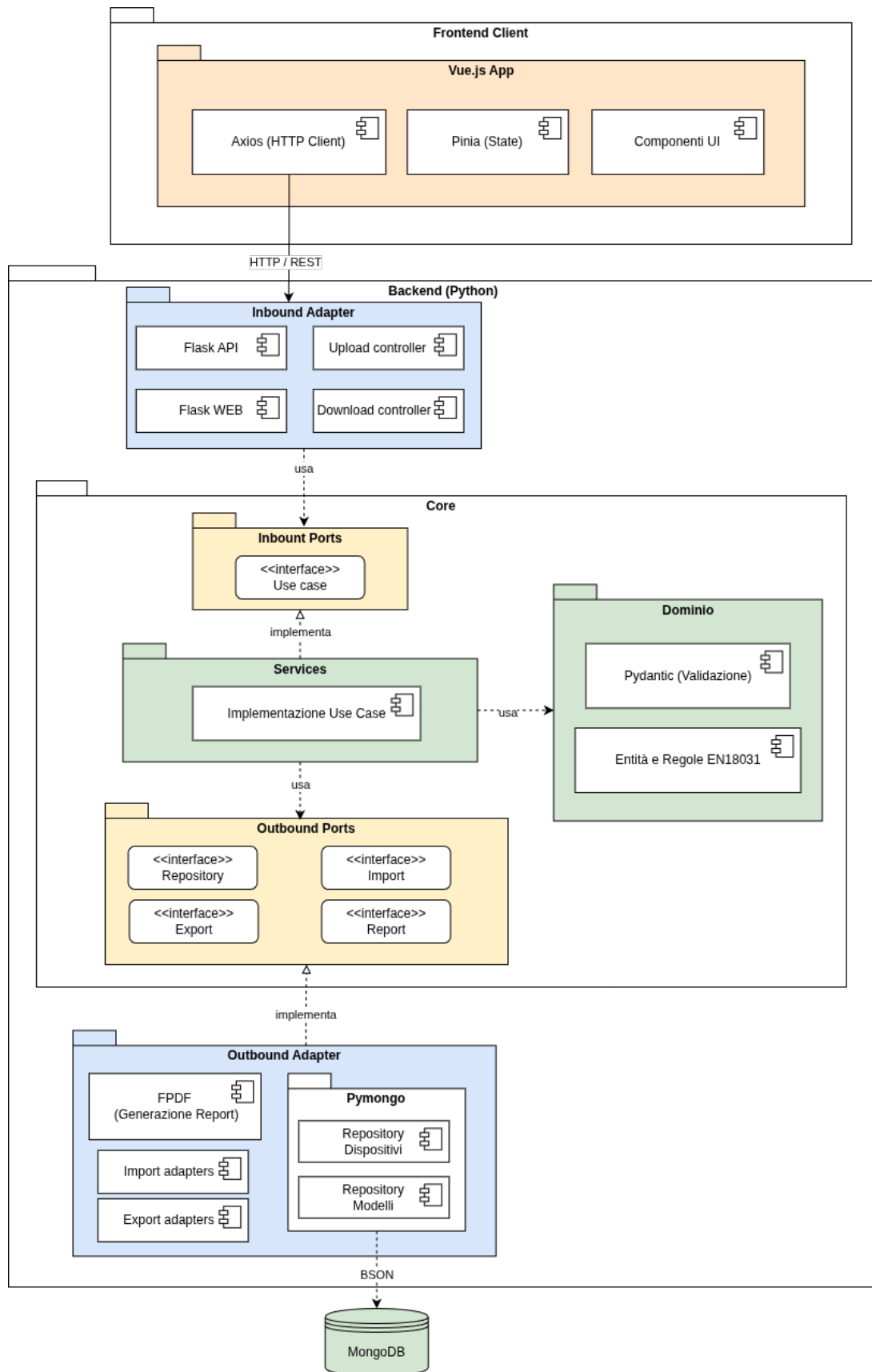
- **Testabilità:** l'assenza di dipendenze esterne nel livello di Dominio permette di scrivere Unit Test in puro Python in modo semplice e veloce. È possibile testare la logica di business in modo isolato, iniettando mock o stub per simulare le interfacce, senza la necessità di avviare il server Flask o connettersi al database MongoDB.
- **Divisione del nucleo:** il nucleo dell'applicazione è totalmente disaccoppiato dall'infrastruttura. Essendo ignaro del livello di presentazione (l'interfaccia utente in HTML, CSS e Vue.js) e del livello di persistenza (il database NoSQL MongoDB), il sistema garantisce un'alta tolleranza ai cambiamenti. È possibile sostituire o aggiornare le tecnologie esterne senza dover alterare la logica di business.
- **Sviluppo parallelo:** la rigorosa definizione dei contratti di comunicazione (le Porte) nelle fasi iniziali di progettazione permette al team di procedere in parallelo. Frontend, backend e integrazione con il database possono essere sviluppati simultaneamente da membri diversi del gruppo, riducendo i colli di bottiglia.
- **Inversione delle dipendenze:** l'architettura applica rigorosamente questo principio: è il Dominio a dettare i contratti (le interfacce) a cui i servizi esterni devono sottostare, e non viceversa. In questo modo, il nucleo dell'applicazione mantiene il controllo assoluto del flusso ed è protetto dai potenziali effetti collaterali (side effects) derivanti dall'infrastruttura.

3.2.3) Limiti dell'architettura

Gli aspetti negativi di questa scelta sono:

- **Ripida curva di apprendimento:** l'architettura esagonale richiede una profonda comprensione dei principi SOLID, in particolare la Dependency Injection e usare questo pattern per la prima volta richiede profondo studio. Questo rischio sarà mitigato da una precisa fase di progettazione che semplificherà la codifica.
- **Elevato overhead iniziale:** La ferrea separazione dei livelli impone la stesura di un'abbondante quantità di codice infrastrutturale (boilerplate). Risulta necessario definire contratti astratti (Porte), implementazioni concrete (Adattatori) e orchestratori (Servizi), allungando i tempi di sviluppo nelle prime fasi del progetto. Il gruppo ha tuttavia accettato questo costo iniziale, ritenendolo un investimento necessario a fronte del drastico abbattimento dei futuri costi di manutenzione e della massima testabilità garantita al nucleo applicativo.

3.2.4) Diagramma dei package



Il diagramma illustra l'organizzazione logica del sistema Automated EN18031 Compliance Verification, fondata sull'architettura esagonale. Il sistema è strutturalmente ripartito in un livello di presentazione esterno (Frontend Client), sviluppato tramite il framework Vue.js, e un nucleo applicativo (Backend), implementato in Python.

Per garantire una rigorosa separazione delle responsabilità e il pieno rispetto del principio di Inversione delle Dipendenze, il Backend si articola nei seguenti livelli tipici dell'architettura esagonale:

1. **Adapters:** Costituisce il confine tecnologico del sistema, responsabile della comunicazione con l'esterno. Si divide in:
 - **Inbound Adapters** (Driving Adapters): Agiscono da punto di ingresso per il sistema. Intercettano gli input esterni (es. le richieste HTTP/REST inviate dal Frontend), ne effettuano il parsing e traducono tali direttive in invocazioni dirette verso le Inbound Ports, senza mai accedere alla logica di business.
 - **Outbound Adapters** (Driven Adapters): Rappresentano le tecnologie concrete di uscita (es. il driver MongoDB o le librerie di generazione PDF). Il loro compito è implementare materialmente le interfacce definite nel livello Outbound Ports, traducendo i comandi astratti in operazioni specifiche per l'infrastruttura.
2. **Core** (Nucleo Applicativo): È il cuore del software, totalmente agnostico e privo di dipendenze verso framework web o database. Al suo interno è stratificato in:
 - **Ports:** Funge da barriera di isolamento. Definisce le interfacce in puro Python, disaccoppiando la tecnologia dall'applicazione:
 - **Inbound Ports** (Primary Ports): Definiscono le interfacce (i contratti) relative ai Casi d'Uso (Use Cases) che il sistema offre. Specificano «cosa» il sistema è in grado di fare, permettendo agli Inbound Adapters di invocare le operazioni senza dover conoscere l'implementazione sottostante.
 - **Outbound Ports** (Secondary Ports): Definiscono i contratti astratti di cui il dominio ha bisogno per comunicare con l'esterno (ad esempio, le interfacce del Repository Pattern per l'accesso ai dati). Queste porte vengono usate dal livello Services e implementate dagli Outbound Adapters.
 - **Services:** Costituiscono il livello di orchestrazione. Implementano concretamente i Casi d'Uso definiti nelle Inbound Ports, coordinano il flusso di esecuzione richiamando le entità di dominio, e si avvalgono delle Outbound Ports per la persistenza dei dati.
 - **Domain:** Incapsula le regole di business pure e la logica della norma EN18031. Contiene la modellazione formale delle entità e la validazione strutturale dei dati (supportata da Pydantic), operando in totale e completo isolamento.

3.3) Architettura di Deployment

Il sistema adotta un'architettura a Monolite Modulare.

L'intera applicazione è concepita, sviluppata e rilasciata come un'unica unità eseguibile.

3.3.1) Motivazioni della scelta

Questa scelta è coerente con lo sviluppo di una web app locale.

In questo contesto un'architettura a monolite modulare, rispetto all'architettura a microservizi, offre i seguenti vantaggi:

- **Semplicità di Deployment:** tutte le funzionalità sono contenute nella singola unità operativa;
- **Latenze ridotte:** lo scambio di informazioni avviene completamente in locale.

Non si è optato per una architettura monolitica tradizionale in cui spesso il codice è fortemente accoppiato.

La rigorosa divisione in moduli logici isolati permette di coniugare la semplicità di rilascio dell'applicazione come singola unità con alcuni dei vantaggi organizzativi tipici delle architetture a microservizi:

- **Separazione delle responsabilità** tra i vari ambiti del dominio.
- **Semplicità di sviluppo e manutenibilità** a lungo termine.
- **Facilità di testing**, permettendo di collaudare i singoli moduli in totale isolamento.

3.3.2) Realizzazione Pratica e Topologia

Per la distribuzione viene usato Docker come tool di containerizzazione e Docker Compose come tool di orchestrazione.

1. Nodi di Esecuzione:

- **Browser dell'utente**, l'interfaccia utente è realizzata tramite browser;
- **Server Backend**, contiene la core logic dell'applicazione e risponde alle richieste dell'utente;
- **Database MongoDB**, sistema di permanenza;

2. Packaging e isolamento. L'infrastruttura è orchestrata tramite Docker Compose e prevede i seguenti container:

• Web-app

Contiene l'ambiente Python, il framework Flask, gli asset statici del frontend e altre dipendenze¹

• Mongoddb

Basato sull'immagine ufficiale di MongoDB. Per prevenire la perdita dei dati questo container è agganciato a un Docker Volume locale.

¹Vedi sezione [Tecnologie - Backend](#)

3. Comunicazioni e protocolli di rete

I nodi comunicano attraverso i seguenti canali e protocolli:

- **Tra Browser e Backend:**

La comunicazione avviene in chiaro tramite protocollo HTTP/REST

- Tra Backend e Database: Flask comunica con MongoDB utilizzando il protocollo binario proprietario.

La porta del database non è esposta verso il browser né verso l'esterno, garantendo l'integrità dei dati.

4. Gestione del Traffico Web

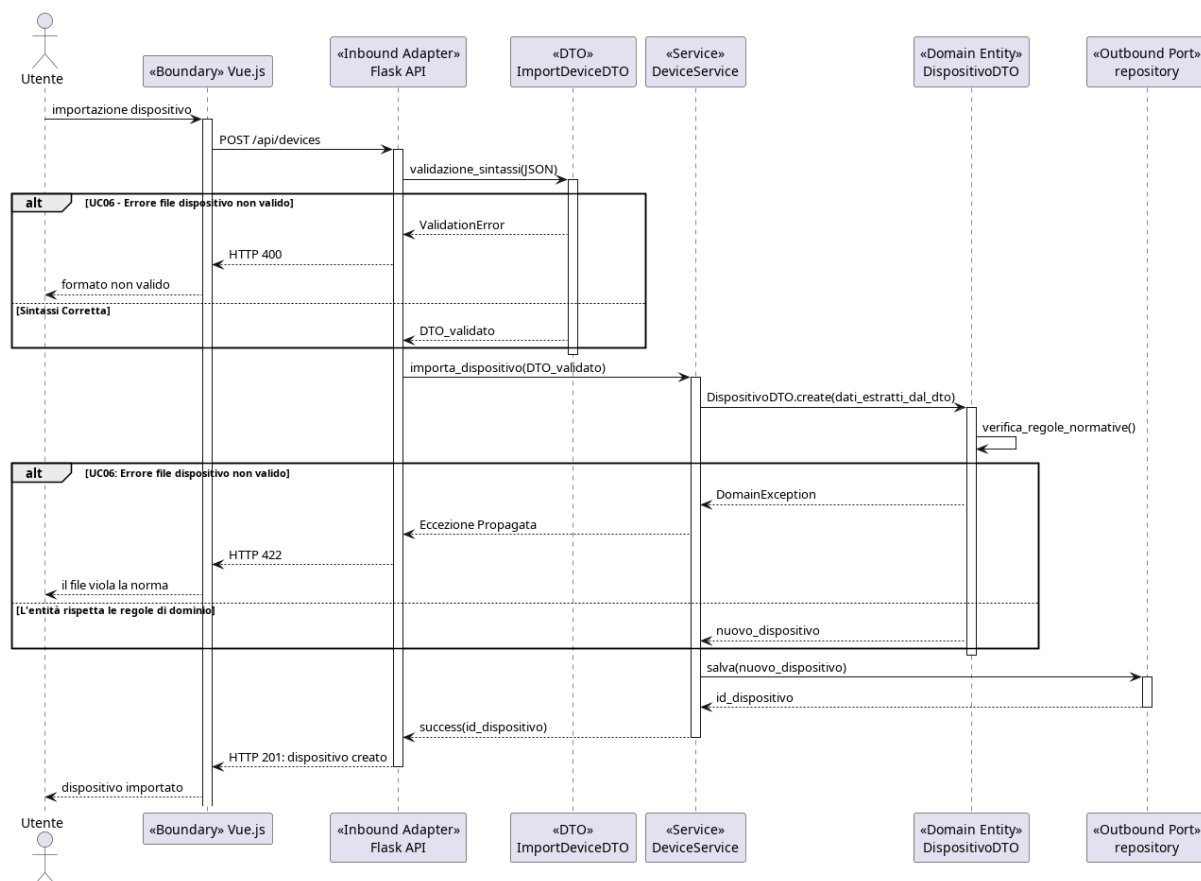
Tra il browser dell'utente e l'applicazione Python è interposto un server **WSGI**.

Questo strato intermedio garantisce stabilità, gestisce la concorrenza delle richieste e le instrada in modo sicuro verso la logica applicativa.

3.4) Diagrammi di sequenza

La seguente sezione illustra il comportamento dinamico del sistema tramite diagrammi di sequenza, focalizzandosi sui casi d'uso di maggiore interesse. Questi modelli descrivono l'ordine cronologico dei messaggi scambiati tra gli attori esterni, i componenti infrastrutturali e il nucleo applicativo.

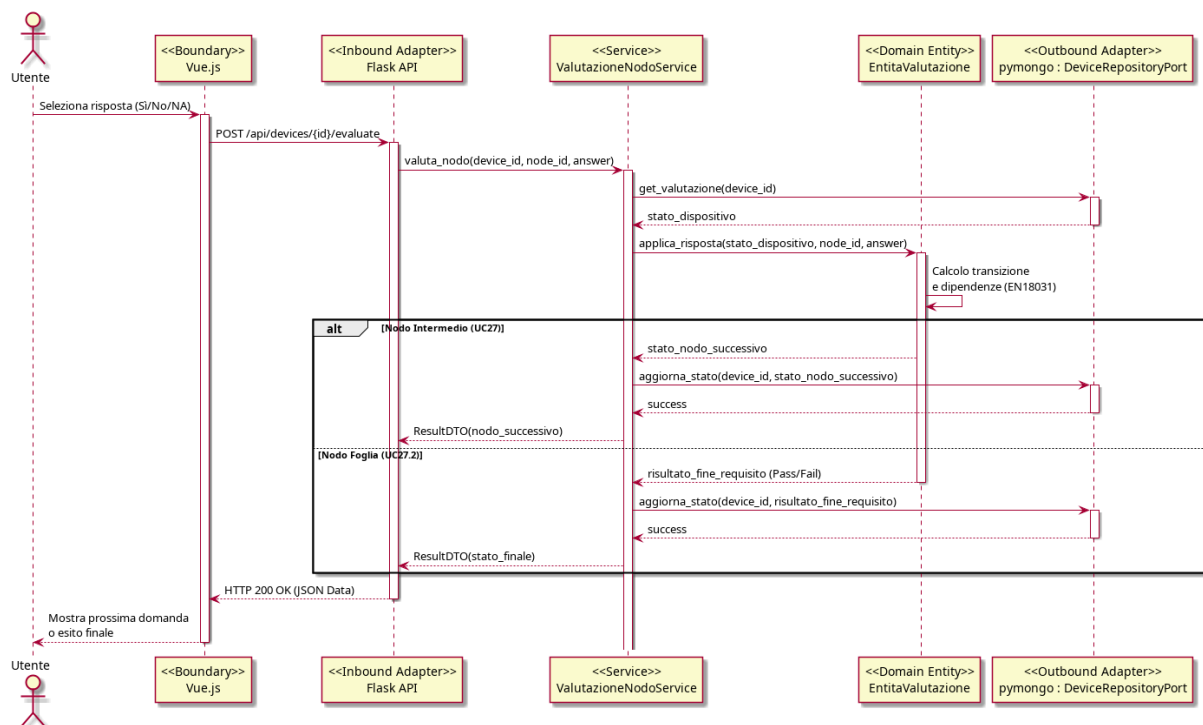
3.4.1) UC05 - Importazione dispositivo



Il diagramma illustra il processo di importazione e validazione strutturale di un dispositivo attraverso i layer dell'Architettura Esagonale. Il flusso adotta un approccio Fail-Fast diviso in due fasi. Inizialmente, l'Adattatore Inbound utilizza un Data Transfer Object (DTO) per eseguire una validazione strutturale sul formato del file in ingresso. Successivamente, il Servizio estrae i dati validati e li passa al Dominio, a cui è delegata esclusivamente la verifica delle regole normative.

Tramite le due aree alt viene modellata la gestione degli errori del caso (UC06): il primo blocco respinge i payload malformati fermandoli al confine del sistema (HTTP 400); il secondo gestisce le violazioni delle regole di business sollevate dal nucleo applicativo (HTTP 422). Solo se l'entità supera entrambi i controlli, il Servizio invoca l'Adattatore di persistenza per il salvataggio e completa l'operazione.

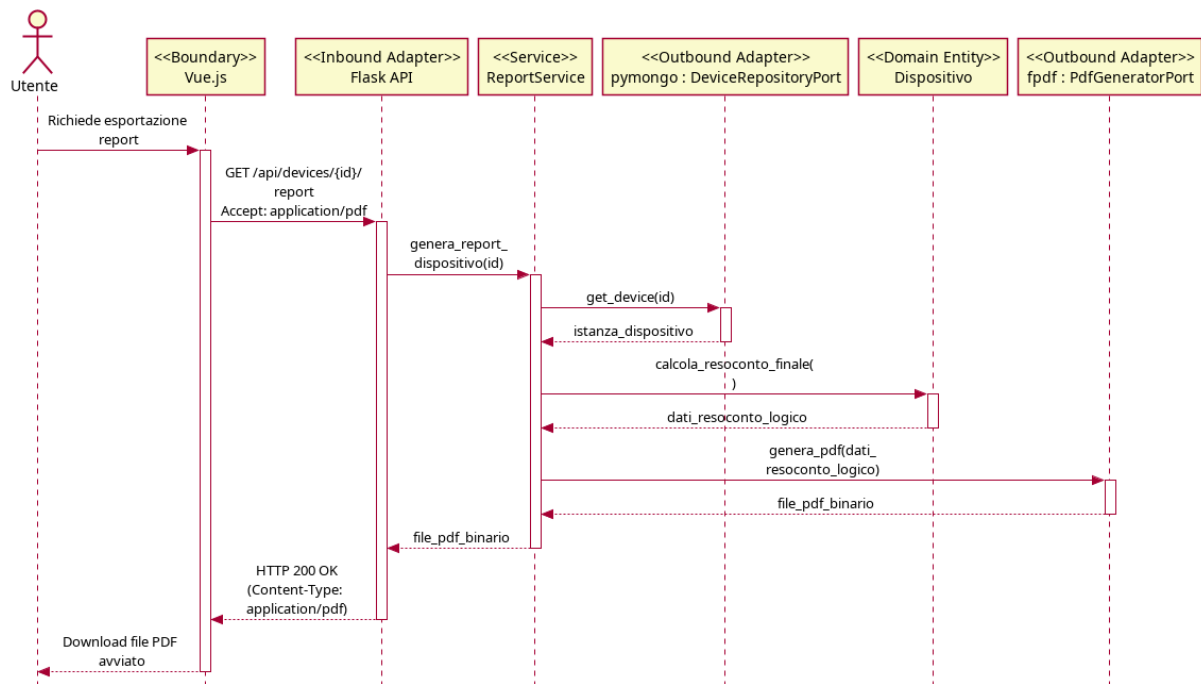
3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato



Il diagramma di sequenza illustra il flusso principale di interazione durante la valutazione di un dispositivo secondo la norma EN18031. Il diagramma evidenzia il rigoroso attraversamento dei layer architetturici, dal Boundary (Vue.js) fino al driver di persistenza (PyMongo).

Degno di nota è l'isolamento del livello Domain: il frontend e gli adapter non conoscono le regole normative. È il Servizio applicativo che recupera lo stato, lo inietta nel Dominio, e si affida al calcolo di quest'ultimo. Tramite l'uso di una sezione alt, il diagramma modella il comportamento polimorfico del flusso: se la risposta porta a un nodo intermedio, il sistema salva il progresso e restituisce la domanda successiva; se la risposta raggiunge una foglia dell'albero normativo, il sistema calcola la conformità finale (Pass/Fail) e chiude lo stato della valutazione.

3.4.3) UC30 - Esportazione report di conformità



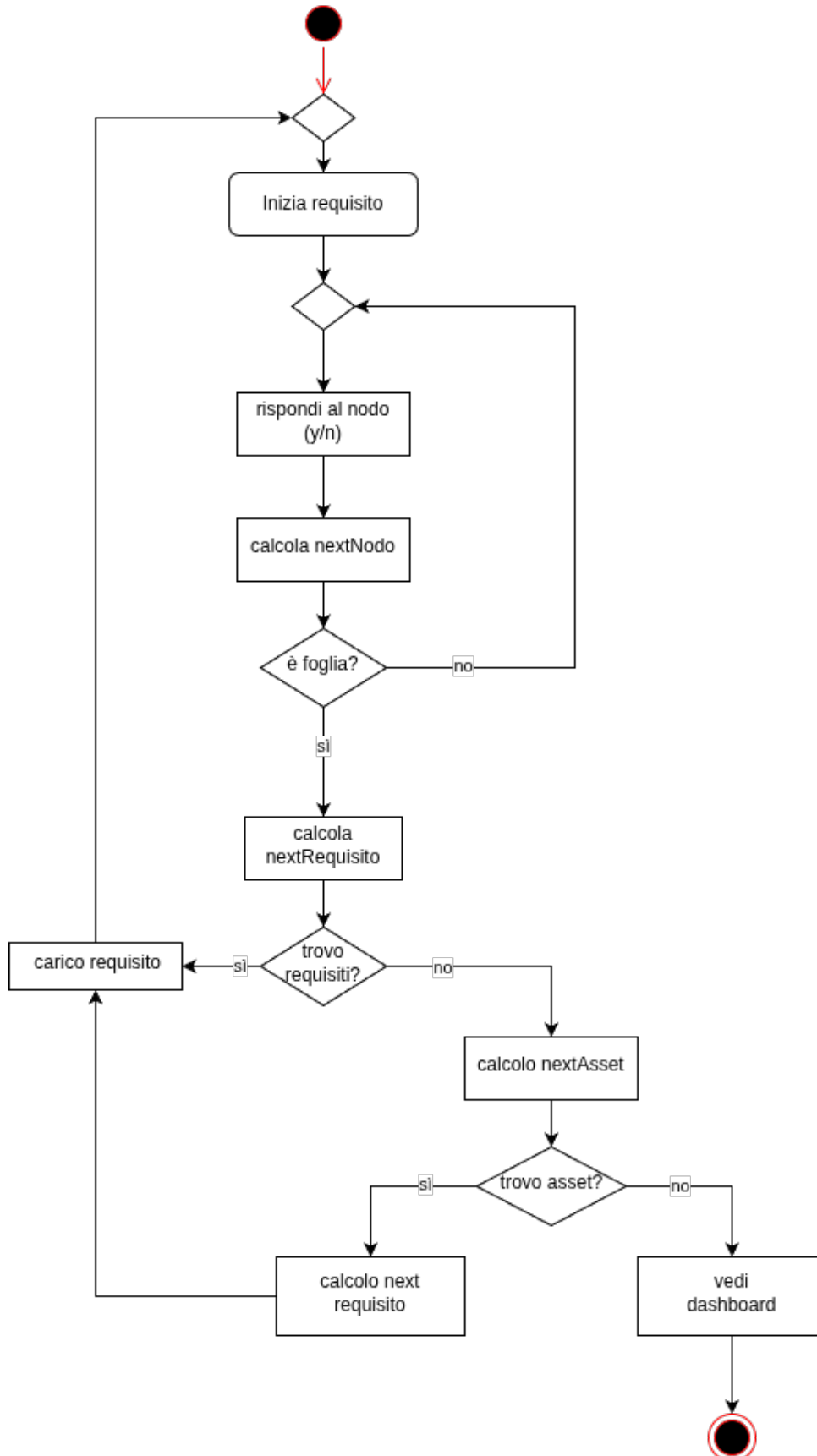
Il diagramma di sequenza illustra il processo di generazione ed esportazione del resoconto finale di conformità per un dispositivo valutato.

Il flusso è innescato da una chiamata HTTP gestita dall'Adattatore Inbound. Il Servizio applicativo avvia il recupero del documento che viene effettuato grazie all'outbound adapter (PyMongo) che estrae i dati dal database.

Una volta recuperato il dispositivo, il Servizio invoca l'aggregazione dei verdeti sul Dominio, il quale restituisce una struttura dati esclusivamente logica (Pass/Fail/NA) senza possedere alcuna conoscenza del rendering finale. Per la creazione del file fisico il Servizio usa la porta di generazione del pdf che traduce i dati puri in un layout grafico, restituendo il file che arriverà all'utente.

3.5) Diagrammi di attività

3.5.1) Navigazione degli alberi



Il diagramma di attività illustra l'algoritmo di navigazione dell'albero normativo.

Il flusso si basa su un ciclo continuo il cui innesco principale è la risposta dell'utente a uno specifico nodo (Sì/No). Dopo l'inserimento dell'input, il sistema calcola il nodo successivo e ne verifica la natura tramite un blocco decisionale («è foglia?»):

Se il nodo non è una foglia (nodo intermedio), il flusso torna indietro per sottoporre all'utente la nuova domanda appena calcolata.

Se il nodo è una foglia (ramo di valutazione concluso), il sistema innesca la logica di avanzamento gerarchico.

La fase di avanzamento procede per livelli. Dapprima, il sistema calcola e verifica se vi sono ulteriori requisiti da valutare per l'asset corrente («trovo requisiti?»). In caso positivo, il nuovo requisito viene caricato e il ciclo di domande riparte dall'inizio. In caso negativo, il sistema sale di livello verificando l'esistenza di ulteriori asset non ancora esaminati nel dispositivo («trovo asset?»). Se viene individuato un nuovo asset, ne vengono calcolati i relativi requisiti, che vengono caricati per riavviare la compilazione.

L'algoritmo fuoriesce da questo ciclo annidato solo ed esclusivamente quando sia i requisiti sia gli asset del dispositivo sono stati completamente esauriti. In questo scenario conclusivo, il sistema torna alla pagina di dashboard.

3.6) Schema dati

In questa sezione viene illustrata l'organizzazione logica dei dati all'interno del database MongoDB.

3.6.1) Scelte Architetture e Formalismo

A differenza dei sistemi relazionali basati su tabelle, MongoDB è un database **NoSQL orientato ai documenti** con struttura *schema-flexible*. Questa scelta architeturale è ideale per gestire strutture dati gerarchiche e dinamiche (come gli alberi decisionali), evitando il sovraccarico computazionale derivante da query di JOIN complesse.

Per la modellazione visiva non si utilizzano diagrammi Entity-Relationship, in quanto non idonei ai database documentali.

Si adotta invece il formalismo di Hackolade (consultabile al link: https://hackolade.com/schemas/Yelp_Challenge_dataset_documentation.html), che descrive chiaramente la gerarchia dei campi, i tipi di dato e le nidificazioni.

Nota sulla validazione dei dati: Benché i documenti vengano illustrati con tipi logici (incluso il tipo `enum` per chiarezza espositiva), la validazione strutturale di base sarà garantita dall'adapter predisposto alle comunicazioni col database

Le due *collection* principali del database sono:

- **Collection «Modelli»:** Catalogo di sola lettura dei template. Contiene le definizioni degli standard, le versioni e la logica degli alberi decisionali.
- **Collection «Dispositivi»:** Registro dinamico dei device censiti. Memorizza lo stato delle valutazioni e il riferimento puntuale al modello normativo applicato.

3.6.2) Schema Dati Dispositivo

Il documento Dispositivo è l'entità centrale del sistema. La sua struttura gerarchica rispecchia la scomposizione fisica e funzionale dell'oggetto analizzato in tre blocchi logici:

1. **Header Identificativo (Root):** Informazioni anagrafiche, tecniche e puntatore al modello normativo.
2. **Lista Asset (Nodi Figli):** Array di sub-documenti che modellano le singole componenti o funzionalità del dispositivo.
3. **Registro Valutazioni (Nodi Terminali):** Dizionario delle risposte fornite per i requisiti applicabili a ciascun asset.

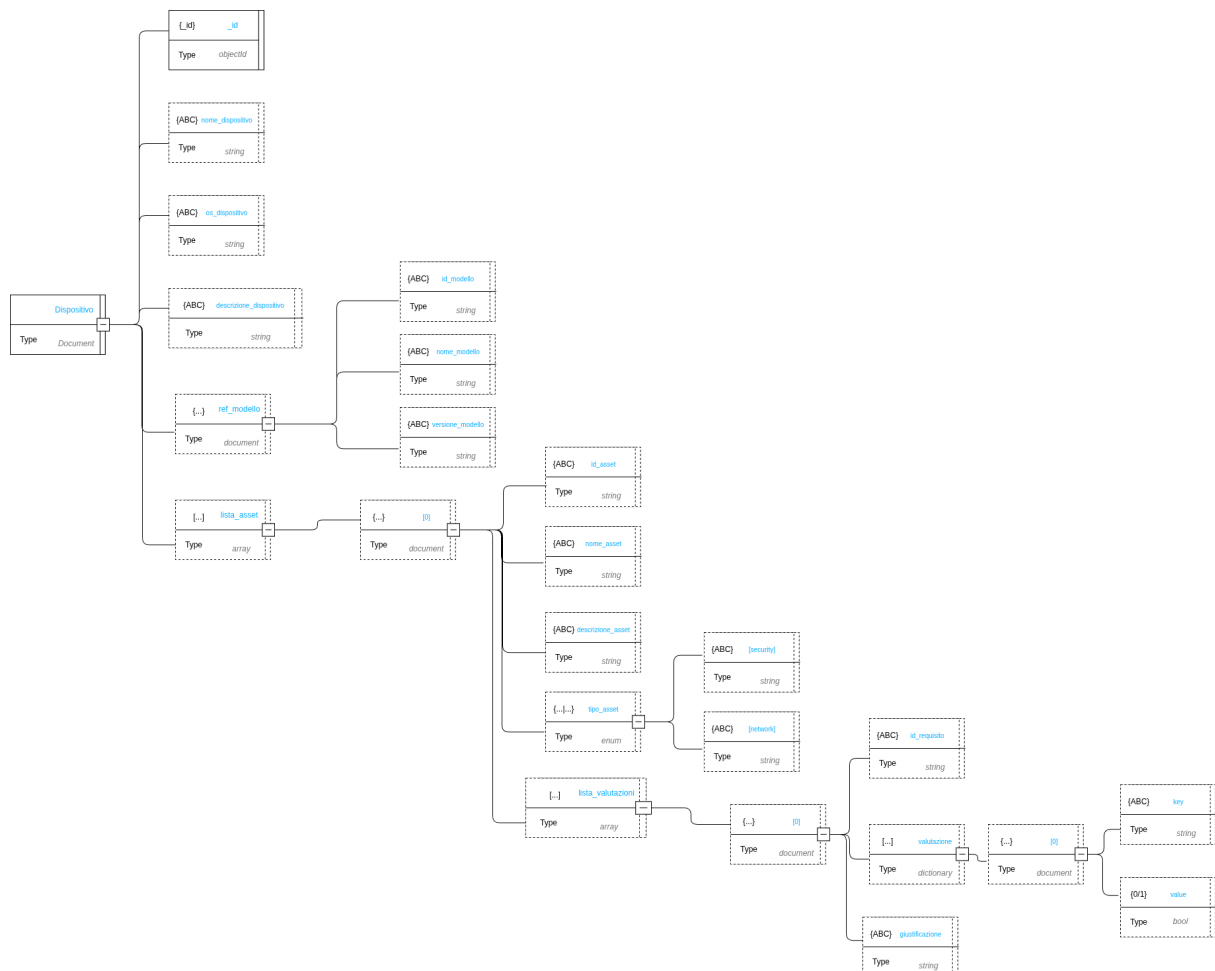


Figura 1: Schema logico: Documento Dispositivo

Root --- Dispositivo

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco generato dal database.
<code>nome_dispositivo</code>	string	Nome assegnato dall'utente al dispositivo. Valore obbligatorio.
<code>os_dispositivo</code>	string	Sistema operativo o firmware installato.
<code>descrizione_dispositivo</code>	string	Testo libero descrittivo del dispositivo.
<code>ref_modello</code>	document	Metadati del modello di riferimento usato per la valutazione (<code>id_modello</code> , <code>nome_modello</code> , <code>versione_modello</code>).
<code>lista_asset</code>	array	Elenco degli asset associati al dispositivo. Può essere inizializzato come array vuoto <code>[]</code> .

Tabella 3: Schema dati: Root — Dispositivo

Sub-documento --- ref_modello

Campo	Tipo BSON	Descrizione
<code>id_modello</code>	string	Identificativo del modello di riferimento nel sistema esterno.
<code>nome_modello</code>	string	Nome leggibile del modello.
<code>versione_modello</code>	string	Versione del modello applicata alla valutazione.

Tabella 4: Schema dati: Sub-documento — ref_modello

Sub-documento --- lista_asset[N]

Campo	Tipo BSON	Descrizione
<code>id_asset</code>	string	ID applicativo per l'identificazione univoca dell'asset nel dispositivo.
<code>nome_asset</code>	string	Nome descrittivo dell'asset.
<code>descrizione_asset</code>	string	Testo libero descrittivo dell'asset.
<code>tipo_asset</code>	enum	Categoria logica dell'asset. Valori ammessi: <code>security</code> , <code>network</code> .
<code>lista_valutazioni</code>	array	Elenco delle valutazioni fornite per questo asset. Può essere vuoto <code>[]</code> .

Tabella 5: Schema dati: Sub-documento — lista_asset[N]

Sub-documento --- lista_valutazioni[N]

Campo	Tipo BSON	Descrizione
id_requisito	string	Riferimento al codice del requisito presente nel modello.
valutazione	dictionary	Mappa chiave-valore delle risposte. La chiave (string) corrisponde al codice del nodo decisionale, il valore (enum) rappresenta l'esito logico: PASS, FAIL, NA (Not Applicable).
giustificazione	string	Testo descrittivo della motivazione. Obbligatorio a livello applicativo in caso di esito FAIL o NA.

Tabella 6: Schema dati: Sub-documento — lista_valutazioni[N]

3.6.3) Schema Dati Modello

La collection «Modelli» contiene la definizione strutturale degli standard normativi. A differenza del Dispositivo, orientato allo stato, il Modello descrive la logica di valutazione. Si suddivide in:

- **Metadati:** Versionamento e identificazione.
- **Albero dei Requisiti:** Struttura che definisce il testo della norma e la logica decisionale per guidare l'utente alla valutazione finale.

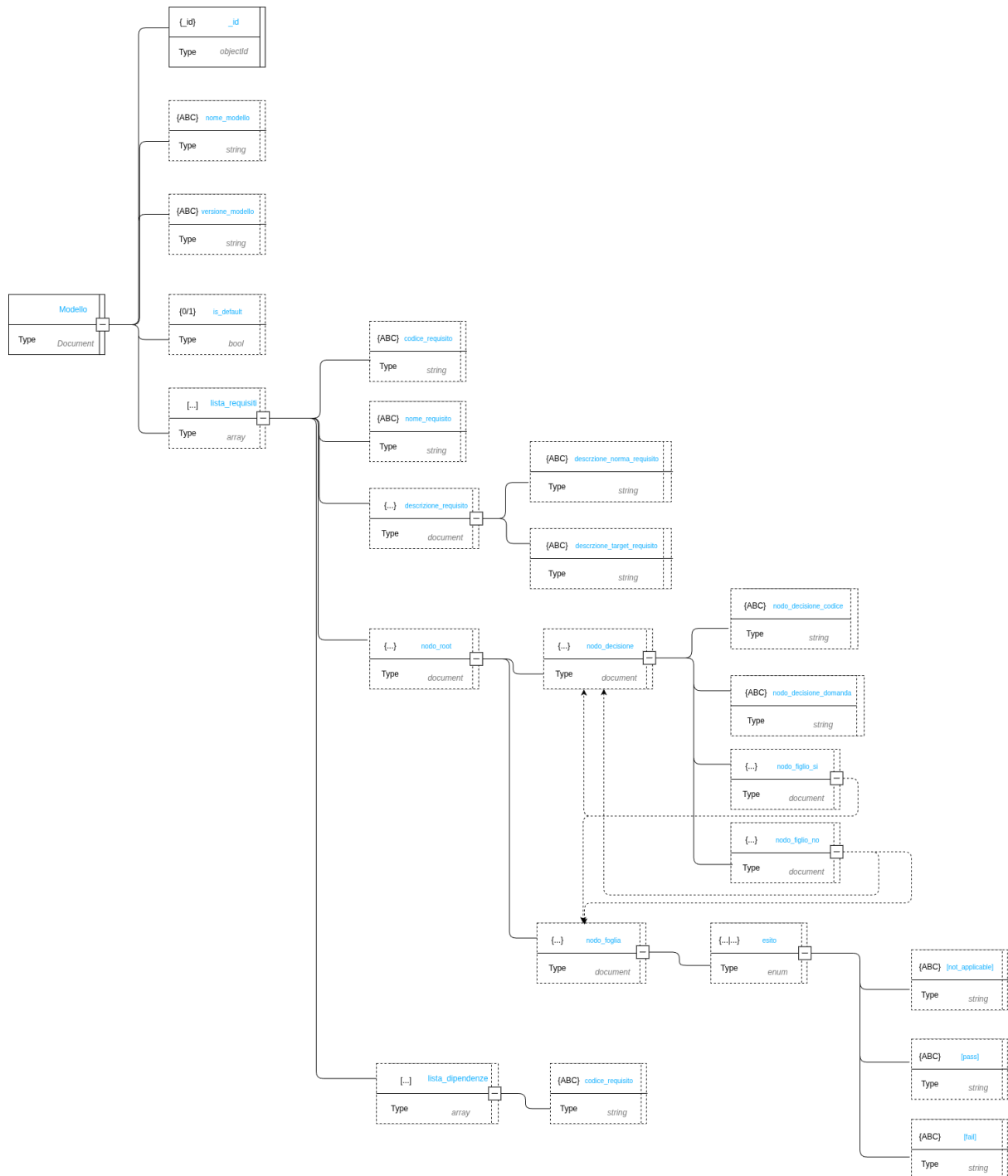


Figura 2: Schema logico: Documento Modello

Root --- Modello

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco del modello.
<code>nome_modello</code>	string	Nome dello standard normativo.
<code>versione_modello</code>	string	Identificativo della versione del modello.
<code>is_default</code>	bool	Flag che indica se il modello è quello predefinito per nuove valutazioni.
<code>lista_requisiti</code>	array	Elenco dei requisiti che compongono la normativa.

Tabella 7: Schema dati: Root — Modello

Sub-documento --- lista_requisiti[N]

Campo	Tipo BSON	Descrizione
<code>codice_requisito</code>	string	Identificativo testuale univoco del requisito.
<code>nome_requisito</code>	string	Titolo sintetico del requisito.
<code>descrizione_requisito</code>	document	Testo descrittivi del contesto normativo (<code>descrizione_norma</code> , <code>descrizione_target</code>).
<code>nodo_root</code>	document	Punto d'ingresso dell'albero decisionale per questo requisito.
<code>lista_dipendenze</code>	array	Elenco di codici di requisiti propedeutici da soddisfare prima della valutazione.

Tabella 8: Schema dati: Sub-documento — lista_requisiti[N]

Sub-documento --- Nodo_decisione

Campo	Tipo BSON	Descrizione
<code>nodo_decisione_codice</code>	string	Codice identificativo del nodo logico.
<code>nodo_decisione_domanda</code>	string	Testo della domanda posta all'utente.
<code>nodo_figlio_si</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta affermativa.
<code>nodo_figlio_no</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta negativa.

Tabella 9: Schema dati: Sub-documento — Nodo_decisione

Sub-documento --- Nodo_foglia

Campo	Tipo BSON	Descrizione
<code>esito</code>	enum	Stato terminale del percorso logico. Valori ammessi: <code>pass</code> , <code>fail</code> , <code>not_applicable</code> .
<code>not_applicable</code>	string	Testo esplicativo in caso di non applicabilità.
<code>pass</code>	string	Messaggio di conferma del soddisfacimento.
<code>fail</code>	string	Messaggio di errore o indicazioni di mitigazione.

Tabella 10: Schema dati: Sub-documento — Nodo_foglia

3.6.4) Gestione degli Esiti e Storicità

La separazione netta tra Modello e Dispositivo ottimizza la memorizzazione e la gestione dei dati portando i seguenti vantaggi:

- **Zero Ridondanza:** Il documento Dispositivo salva unicamente il dizionario delle risposte fornite, mentre il documento Modello conserva esclusivamente la struttura statica dell'albero. Questo evita di duplicare l'intera gerarchia normativa per ogni dispositivo.
- **Disaccoppiamento:** Viene lasciato alla logica applicativa il compito di «fondere» i dati, compilando a runtime il template del Modello con le risposte storicizzate nel Dispositivo.
- **Versionamento:** La collection Modelli conserva tutte le versioni rilasciate di uno standard. Poiché il Dispositivo punta esplicitamente a una singola e specifica versione, lo storico delle certificazioni passate rimane inalterato e coerente anche a fronte di futuri aggiornamenti normativi.

4) Design Patterns

4.1) Principi di Design

4.1.1) Dependency Injection

4.1.1.1) Descrizione del pattern

La dependency injection prevede che le dipendenze necessarie a una classe o a un modulo non vengano create internamente, ma fornite dall'esterno.

Generalmente questo avviene alla costruzione, tramite un setter o come parametro di un metodo.

4.1.1.2) Motivazioni dell'utilizzo del pattern

La dependency injection permette di realizzare la Dependency Inversion e garantisce che le classi di dominio non dipendano mai da servizi secondari, framework o tecnologie esterne.

4.1.1.3) Utilizzo del pattern nel progetto

All'interno del progetto viene usata la Constructor Injection. Il factory di Flask (`create_app()`) funge da **Composition Root**: è l'unico punto centralizzato del sistema a conoscere le implementazioni concrete delle porte e ad occuparsi di iniettarle nei service applicativi.

4.2) Pattern Architetture

4.2.1) Repository

4.2.1.1) Descrizione del pattern

Il Repository è un pattern architetturale che astrae il sistema di persistenza dietro un'interfaccia che simula una collezione in memoria. Chi usa il Repository non conosce i dettagli del database sottostante.

Interagisce solo con metodi semantici come `findById()`, `save()`, `findAll()`.

4.2.1.2) Motivazioni dell'utilizzo del pattern

Senza Repository la logica di business sarebbe mescolata con le query.

Inoltre i test unitari richiederebbero un database.

Con Repository il dominio e i service applicativi dipendono solo dall'interfaccia nei test si usa un mock.

4.2.1.3) Utilizzo del pattern nel progetto

Nell'ambito del progetto viene utilizzata per gestire le comunicazioni con sistema di persistenza.

La lettura delle collection presenti su mongo DB avviene tramite porte dedicate.

DeviceRepositoryPort implementata da MongoDeviceRepository
e ComplianceStandardRepositoryPort implementata da
MongoComplianceStandardRepository

4.2.2) Unit of Work

4.2.2.1) Descrizione del pattern

La Unit of Work è un pattern architetturale che traccia tutte le modifiche effettuate durante un'operazione e le applica al sistema di persistenza in modo atomico.

Se qualcosa va storto durante la scrittura, nessuna modifica parziale viene persistita.

4.2.2.2) Motivazioni dell'utilizzo del pattern

Una sessione di valutazione può coinvolgere decine di modifiche e risposte ai nodi dei decision tree, aggiornamenti di stato, giustificazioni.

Scrivere ogni modifica immediatamente su MongoDB esporrebbe il sistema a stati inconsistenti se l'utente abbandona la sessione a metà.

La Unit of Work garantisce che il documento persistito sia sempre in uno stato coerente.

4.2.2.3) Utilizzo del pattern nel progetto

Nel progetto, questo pattern regola il ciclo di sessione. Il ValutazioneService accumula e gestisce le modifiche di stato direttamente in memoria durante la navigazione.

Il documento su MongoDB non viene mai toccato o aggiornato durante i passaggi intermedi: la scrittura sul database avviene una sola volta, in modo atomico tramite il Repository, solo quando si raggiunge un nodo foglia o quando il progresso intermedio viene esplicitamente confermato.

4.3) Design Patterns Creazionali

4.3.1) Factory

4.3.1.1) Descrizione del pattern

Il pattern Factory è un pattern creazionale che astrae il processo di creazione degli oggetti.

Delega a un componente specifico la responsabilità di creare oggetti complessi, restituendoli attraverso un'interfaccia comune.

4.3.1.2) Motivazioni dell'utilizzo del pattern

In un'architettura esagonale, i layer di dominio devono dipendere da interfacce (Porte) e mai da implementazioni concrete.

Se un service dovesse istanziare direttamente le proprie dipendenze, finirebbe per accoppiarsi a librerie o tecnologie esterne.

Il Factory risolve questo problema incapsulando la conoscenza di «come» e «quale» oggetto concreto creare.

4.3.1.3) Utilizzo del pattern nel progetto

Nel progetto il pattern è applicato a due livelli distinti:

1. **Application Factory:** A livello di framework, la funzione `create_app()` di Flask agisce come macro-factory, assemblando l'applicazione e le sue dipendenze principali all'avvio.
2. **Domain/Service Factory:** A livello applicativo, vengono utilizzate factory specifiche per costruire a runtime le implementazioni corrette di alcune interfacce che richiedono una selezione basata sulle azioni dell'utente (creare il `FileExporter` corrispondente al formato richiesto dall'utente), restituendo ai service oggetti pronti all'uso che rispettano l'interfaccia attesa.

4.4) Design Patterns strutturali

4.4.1) Adapter

4.4.1.1) Descrizione del pattern

L'Adapter è un pattern strutturale che traduce l'interfaccia di un componente esterno nell'interfaccia che il sistema si aspetta.

Permette a due componenti incompatibili di collaborare senza che nessuno dei due debba cambiare la propria interfaccia.

4.4.1.2) Motivazioni dell'utilizzo del pattern

Il dominio definisce le proprie interfacce in termini di concetti di business.

Tramite l'uso sistematico di porte e adapter è possibile ridurre l'accoppiamento permettendo l'aggiornamento parallelo dei sistemi e lasciando all'adapter il compito di tradurre le nuove interfacce

4.4.1.3) Utilizzo del pattern nel progetto

Al fine di implementare correttamente l'architettura esagonale nel sistema backend, il pattern adapter è stato usato in modo sistematico per gestire le comunicazioni verso servizi esterni.

Gli utilizzi principali sono nella comunicazione col database e con i servizi di interpretazione e generazione di file

4.4.2) Facade

4.4.2.1) Descrizione del pattern

Il Facade è un pattern strutturale che fornisce un'interfaccia di alto livello e semplificata a un insieme di componenti complessi o a un intero sottosistema. L'obiettivo è nascon-

dere la complessità interna esponendo al chiamante unicamente i metodi strettamente necessari all'azione richiesta.

4.4.2.2) Motivazioni dell'utilizzo del pattern

In un'architettura a livelli, i Facade evitano che i layer esterni (come i controller o le interfacce utente) debbano orchestrare manualmente le dipendenze e la logica applicativa. Questo permette di mantenere i moduli di ingresso «sottili» e disaccoppiati dalle dinamiche interne del dominio.

4.4.2.3) Utilizzo del pattern nel progetto

Nel progetto, questo pattern si riflette nei Service, come ad esempio il `FileImporterService`. Anche se espone un'interfaccia minimale, agisce da Facade orchestrando diverse operazioni sottostanti: riceve la richiesta, coordina i servizi di estrazione e parsing del file, e istruisce il sistema di persistenza per il salvataggio, nascondendo l'intera complessità della sequenza al chiamante.

4.5) Design Patterns Comportamentali

4.5.1) Strategy

4.5.1.1) Descrizione del pattern

Lo Strategy è un pattern comportamentale che definisce una famiglia di algoritmi intercambiabili, incapsulandoli in classi separate che condividono la stessa interfaccia. Il chiamante seleziona quale algoritmo usare a runtime senza modificare il proprio codice.

4.5.1.2) Motivazioni dell'utilizzo del pattern

Il sistema deve supportare più formati di import ed export — JSON, CSV, XML — e potenzialmente nuovi formati in futuro. Con Strategy supportare un nuovo formato consiste nel creare una nuova classe che implementa lo strategy pattern senza modificare il codice già esistente.

4.5.1.3) Utilizzo del pattern nel progetto

Viene utilizzato nel processo di importazione del file

4.5.2) Template Method

4.5.2.1) Descrizione del pattern

Il Template Method è un pattern comportamentale che definisce lo scheletro di un algoritmo in una classe astratta, delegando alle sottoclassi l'implementazione dei passi specifici.

La sequenza complessiva è fissa; ciò che cambia è il comportamento dei singoli passi.

4.5.2.2) Motivazioni dell'utilizzo del pattern

Tutti i formati di export condividono la stessa sequenza operativa: preparare la struttura, scrivere i dati, finalizzare l'output.

Senza Template Method questa sequenza dovrebbe essere replicata in ogni exporter concreto, introducendo duplicazione e rischio di inconsistenze.

Il Template Method consente di scrivere la sequenza una sola volta nella classe astratta, lasciando alle sottoclassi solo i dettagli specifici del formato.

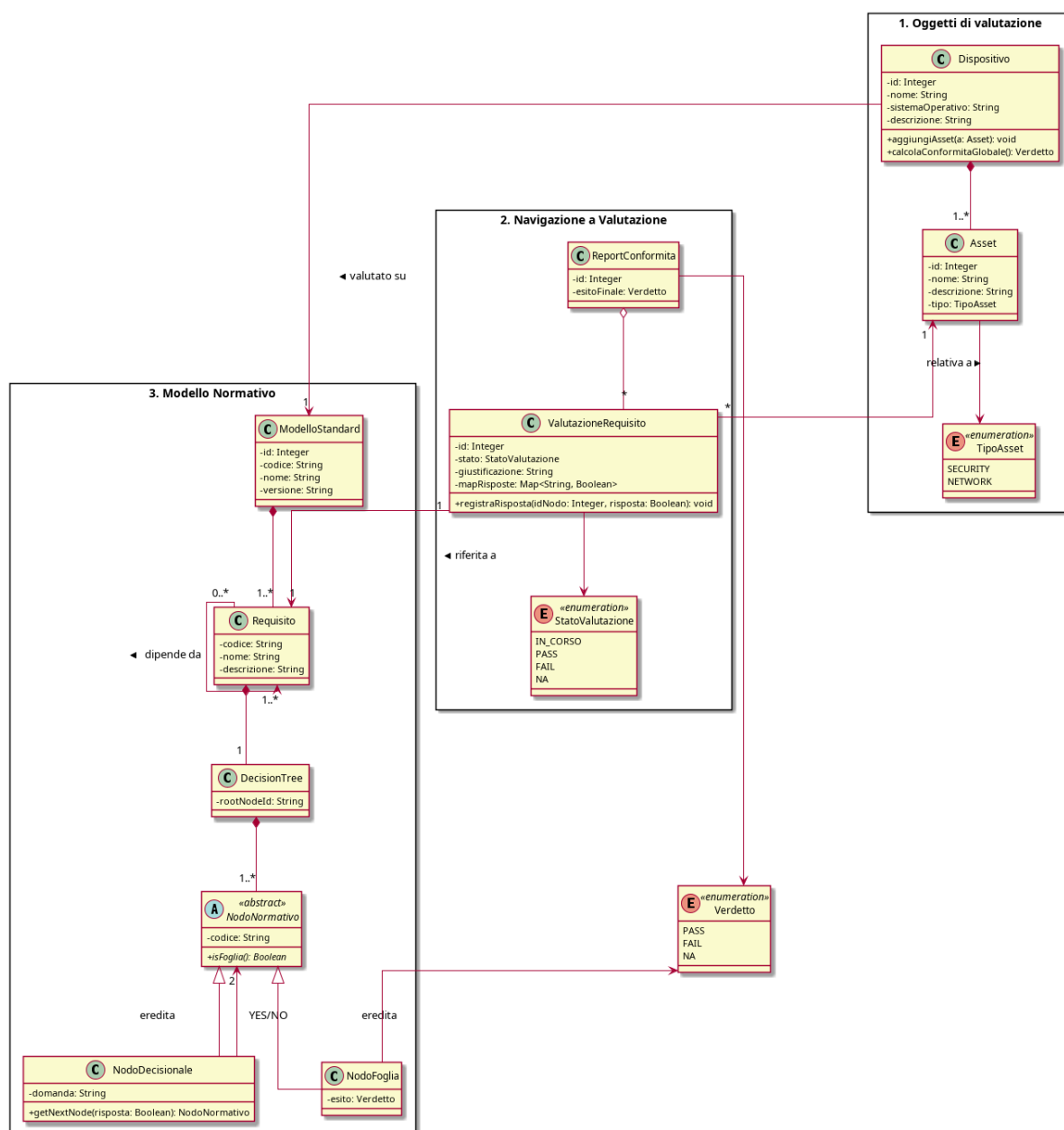
4.5.2.3) Utilizzo del pattern nel progetto

Viene utilizzato durante l'esportazione di informazioni sotto formato di file in diversi formati.

5) Diagrammi delle classi

5.1) Diagramma di dominio

Il diagramma delle classi illustra il Modello di Dominio del nucleo applicativo. Progettato rispettando i principi del Domain-Driven Design e dell'Architettura Esagonale, il modello incapsula esclusivamente la logica di business pura, risultando del tutto agnostico rispetto ai dettagli infrastrutturali.



Per garantire coerenza logica e separazione delle responsabilità, le classi sono state logicamente organizzate in tre macro-package:

1. Oggetti di Valutazione

Questo package modella le entità concrete sottoposte alla valutazione:

- **Dispositivo**: mantiene lo stato globale dell'oggetto in esame e gestisce il ciclo di vita delle sue componenti.
- **Asset**: rappresenta i singoli componenti fisici o logici (classificati tramite l'enumerazione `TipoAsset` in `Security` o `Network`) che costituiscono il dispositivo.

La relazione tra **Dispositivo** e **Asset** è una composizione. La distruzione logica di un dispositivo all'interno del sistema comporta la distruzione dei relativi asset associati.

2. Modello Normativo

Questo package incapsula la struttura formale della norma di riferimento.

- **ModelloStandard** e **Requisito**: Il **ModelloStandard** definisce l'anagrafica della norma e si compone di molteplici **Requisiti**. La relazione riflessiva su **Requisito** («dipende da») permette di modellare i vincoli di dipendenza tra requisiti.
- **DecisionTree**: la struttura dell'albero di valutazione è stata modellata sfruttando il Design Pattern Composite. La classe astratta **NodoNormativo** definisce il contratto base, che viene implementato polimorficamente da:
 - **NodoDecisionale**: nodi intermedi che incapsulano una domanda e definiscono una biforcazione logica (YES/NO) verso i nodi successivi.
 - **NodoFoglia**: nodi terminali del ramo decisionale che emettono un **Verdetto** di conformità (PASS, FAIL, NA) per l'albero a cui appartengono

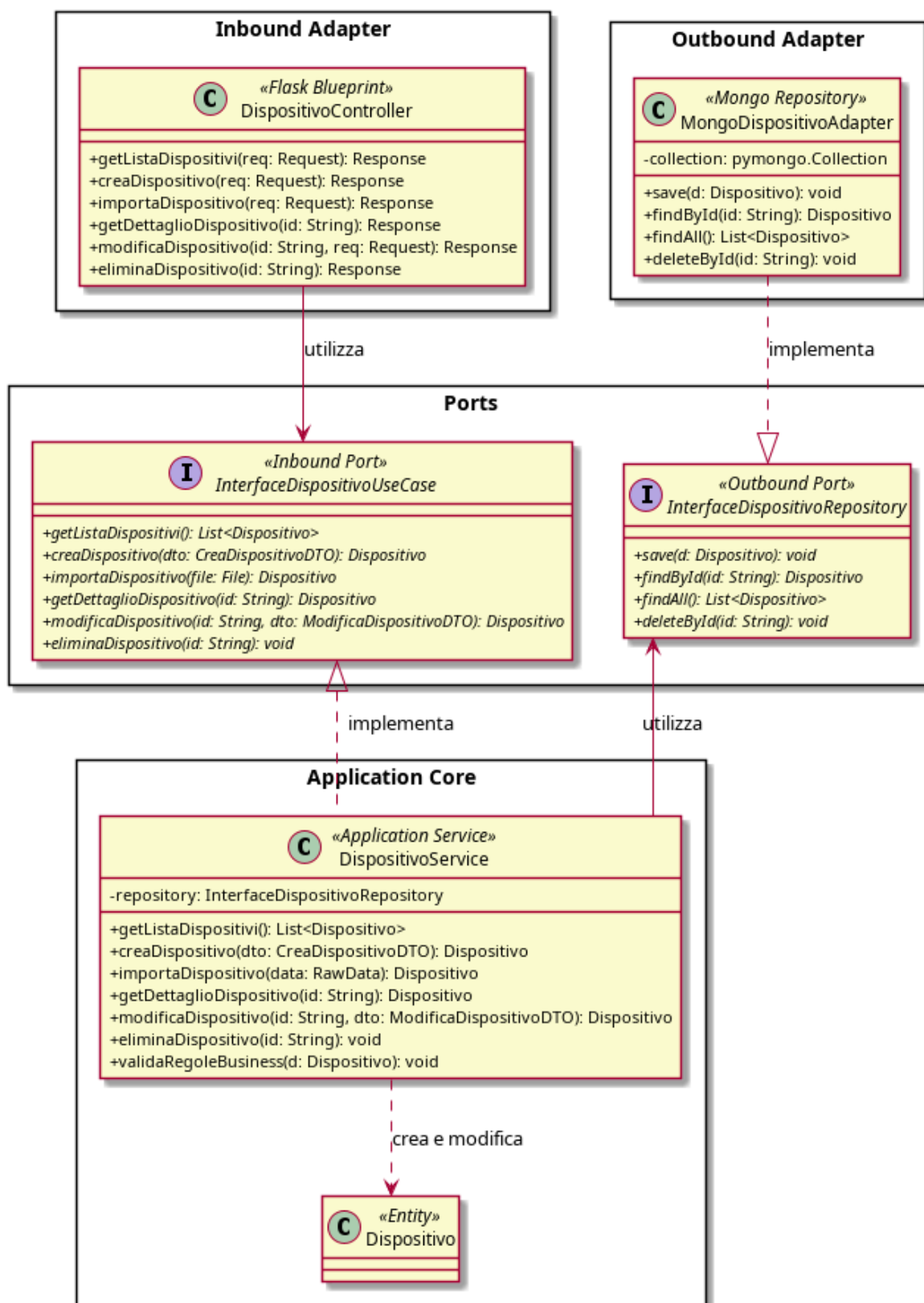
3. Navigazione e Valutazione

Questo package funge da area di interazione tra l'entità fisica e la regola normativa.

- **ValutazioneRequisito**: Agisce come Classe di Associazione tra uno specifico **Asset** e uno specifico **Requisito**. Questa scelta progettuale separa nettamente la definizione della norma dallo stato della valutazione corrente. Contiene una struttura dati (`mapRisposte`) fondamentale per memorizzare le risposte fornite dall'utente durante la navigazione del **DecisionTree**, garantendo la possibilità di sospendere e riprendere la compilazione.
- **ReportConformita**: È legato a **ValutazioneRequisito** tramite una relazione di aggregazione: il report colleziona le singole valutazioni per generare l'esito finale, ma una sua eventuale eliminazione o rigenerazione non invalida i dati delle valutazioni persistite nel sistema.

Viene fatto utilizzo di enumerazioni (`TipoAsset`, `StatoValutazione`, `Verdetto`). Questa scelta impone vincoli di dominio stringenti, garantendo la type-safety ed evitando stati di valutazione non previsti dal capitolato.

5.2) Dispositivo



Il diagramma delle classi illustra la progettazione architetturale per il modulo di Gestione dei Dispositivi.

1. Inbound Adapter

Il pacchetto Inbound Adapter rappresenta il punto di contatto con l'utente. Contiene il `DispositivoController`, sviluppato con il framework Flask. Il suo unico compito è ricevere le richieste HTTP, tradurle in un formato comprensibile al sistema e restituire una risposta web. Questo livello non prende nessuna decisione logica.

2. Application Core e Ports

Al centro del diagramma si trova la logica vera e propria del software. Per proteggere questa parte centrale, essa comunica con l'esterno unicamente tramite delle Porte rappresentate da interfacce:

`InterfaceDispositivoUseCase` (Inbound Port): È l'elenco dei servizi offerti all'utente. Il Controller «utilizza» questa porta per inviare i comandi, senza aver bisogno di sapere come verranno eseguiti.

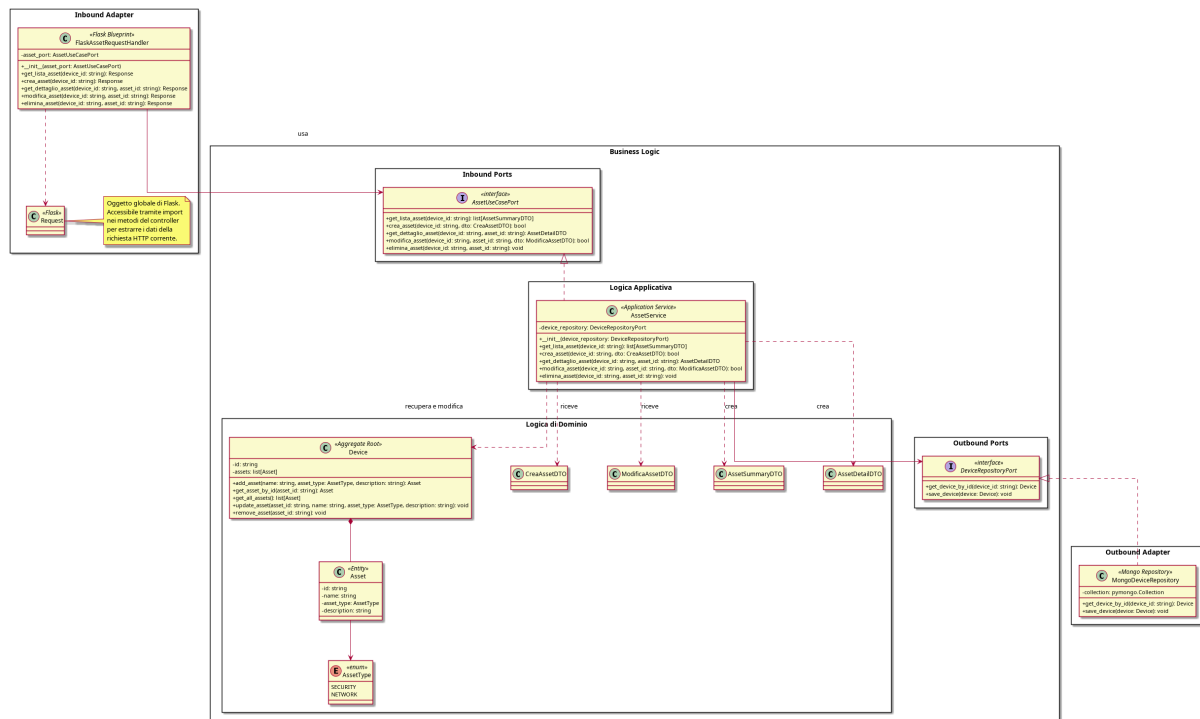
`DispositivoService` (Service): È la classe che svolge il lavoro reale. Riceve i comandi dalla porta Inbound, crea le entità (`Dispositivo`) e verifica che i dati rispettino le regole del progetto tramite un metodo privato dedicato (`validaRegoleBusiness`).

`InterfaceDispositivoRepository` (Outbound Port): Quando il Service ha finito i controlli e deve salvare i dati, non contatta direttamente il database. Usa invece questa porta di uscita, che dichiara solo il bisogno di salvare o leggere un dato, senza specificare la tecnologia.

Outbound Adapter

Il pacchetto Outbound Adapter contiene il `MongoDispositivoAdapter`. Questa classe implementa il contratto richiesto dalla porta in uscita e traduce gli oggetti del programma in documenti fisici salvati su MongoDB.

5.3) Asset



Il diagramma delle classi illustra la progettazione architetturale per il modulo di Gestione degli Asset.

Un Asset rappresenta una componente fisica o logica di un Dispositivo e viene sempre gestito nel contesto del Dispositivo padre, identificato tramite `device_id`.

1. Inbound Adapter

Il pacchetto Inbound Adapter rappresenta il punto di contatto con l'utente. Contiene il flask asset request handler, sviluppato come Blueprint Flask. Il suo unico compito è ricevere le richieste HTTP, tradurle in un formato comprensibile al sistema e restituire una risposta web. Gli endpoint sono annidati gerarchicamente sotto la risorsa Dispositivo, poiché un Asset non esiste in modo autonomo rispetto al Dispositivo padre. Questo livello non prende nessuna decisione logica.

Vengono usati dei DTO specifici per gestire il passaggio dei dati relativi agli asset tra le varie parti del sistema, questo permette al dispositivo di mantenere il possesso esclusivo sugli asset

2. Application Core e Ports

Al centro del diagramma si trova la logica vera e propria del software. Per proteggere questa parte centrale, essa comunica con l'esterno unicamente tramite delle Porte (Interfacce astratte):

AssetUseCasePort (Inbound Port): È l'elenco dei servizi offerti all'utente. Il request handler «utilizza» questa porta per inviare i comandi, senza aver bisogno di sapere come verranno eseguiti.

AssetService (Service): Questa classe realizza le funzioni dalle Inbound Port, crea e modifica le entità Asset, ma sempre utilizzando i metodi della classe Device.

DeviceRepositoryPort (Outbound Port): Quando il Service deve salvare i dati, non contatta direttamente il database. Usa invece questa porta di uscita, che dichiara solo il bisogno di salvare o leggere un dato, senza specificare la tecnologia.

3. Outbound Adapter

Il pacchetto Outbound Adapter contiene il `MongoDeviceRepository`. Questa classe implementa il contratto richiesto dalla porta `DeviceRepositoryPort` e traduce gli oggetti di dominio in operazioni concrete su MongoDB.

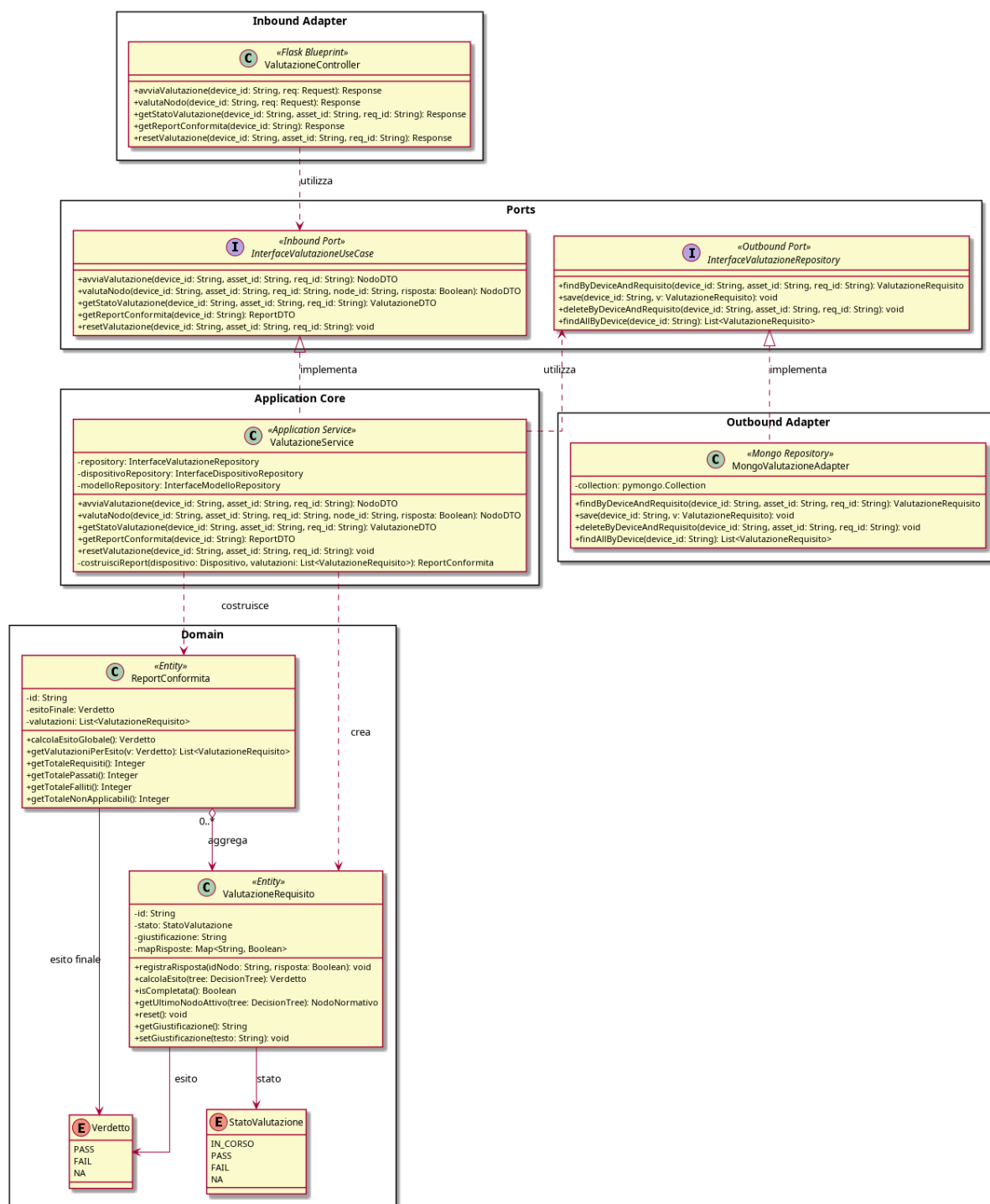
Essendo relazione di composizione tra dispositivo e asset una regola di business, si è preferito mantenere un'unica porta repository e di non permettere la modifica diretta degli asset senza passare dal device, cosa che rischierebbe di violare le regole di business e di causare stati inconsistenti.

Tuttavia questa scelta richiede di non esporre un metodo granulare per il salvataggio delle modifiche agli asset sul sistema di persistenza.

Questo richiederà un'ottimizzazione da parte del repository o causerà un costo in termini di performance.

È in valutazione anche l'introduzione di un'unità di dominio `registroModifiche`, il cui compito è di tracciare le modifiche e permettere a servizi esterni di replicarle.

5.4) Valutazione



Il diagramma delle classi illustra la progettazione architetturale per il modulo di Gestione delle Valutazioni dei Requisiti. Questo modulo è il cuore operativo del sistema: implementa la navigazione degli alberi decisionali della norma EN 18031 e la produzione degli esiti di conformità (PASS/FAIL/NA) per ciascun requisito applicato a un asset.

- Inbound Adapter** Il pacchetto Inbound Adapter contiene il `ValutazioneController`, sviluppato come Blueprint Flask. Il suo unico compito è

ricevere le richieste HTTP, tradurle in un formato comprensibile al sistema e restituire una risposta web. Questo livello non prende nessuna decisione logica; delega interamente il controllo alle Inbound Ports. Gli endpoint esposti corrispondono alle operazioni principali del ciclo di vita di una valutazione: avvio, progressione nodo per nodo, lettura dello stato corrente, reset e richiesta del report finale.

2. Application Core e Ports

Al centro del diagramma si trova la logica vera e propria del software. Per proteggere questa parte centrale, essa comunica con l'esterno unicamente tramite delle Porte (Interfacce astratte):

InterfaceValutazioneUseCase (Inbound Port): Definisce il contratto dei casi d'uso offerti all'esterno. Il Controller invoca questa porta senza conoscere l'implementazione sottostante. I metodi principali riguardano l'avvio (`avviaValutazione`), la progressione nodo per nodo (`valutaNodo`), la lettura dello stato corrente (`getStatoValutazione`), il reset (`resetValutazione`) e la generazione del report finale (`getReportConformita`).

ValutazioneService (Service): È la classe che svolge il lavoro reale. Implementa `InterfaceValutazioneUseCase` e orchestra l'intero flusso di navigazione degli alberi decisionali. In particolare:

- recupera il contesto di valutazione (dispositivo, modello normativo, stato corrente) tramite i repository iniettati (`dispositivoRepository` e `modelloRepository`) esposti come Outbound Ports appartenenti ai rispettivi moduli;
- delega al Domain il calcolo della transizione di stato e della logica normativa EN 18031;
- utilizza un metodo privato `caricaContestoValutazione` per centralizzare il caricamento ed evitare duplicazione tra le operazioni.

InterfaceValutazioneRepository (Outbound Port): Definisce il contratto di persistenza specifico per le valutazioni che il Service utilizza. Astrae completamente la tecnologia di storage: il nucleo non conosce MongoDB. I metodi esposti comprendono la ricerca per dispositivo e requisito, il salvataggio, la cancellazione e il recupero completo per dispositivo.

3. Domain

Il livello Domain incapsula le entità e le regole di business pure.

- **ValutazioneRequisito:** è la classe di associazione centrale tra un Asset e un Requisito normativo (già visibile nel diagramma di dominio alla Sezione 5.1). Memorizza nella mappa `mapRisposte` le risposte booleane date dall'utente per ciascun nodo dell'albero, permettendo la sospensione e la ripresa della compilazione. Il metodo `calcolaEsito` delega al `DecisionTree` la restituzione del verdetto

finale percorrendo il cammino registrato. Il metodo `getUltimoNodoAttivo` determina a quale nodo dell'albero l'utente deve essere reindirizzato in caso di ripresa.

- **ReportConformita**: aggrega una collezione di `ValutazioneRequisito` relative a un intero `Dispositivo`. La relazione è di aggregazione e non di composizione: l'eventuale rigenerazione o cancellazione di un report non invalida le valutazioni persistite nel sistema. Offre metodi per calcolare l'esito globale e per interrogare la distribuzione dei verdetti (totale PASS, FAIL, NA).
- **StatoValutazione** e **Verdetto**: due enumerazioni che impongono vincoli di dominio stringenti sullo stato del processo di valutazione e sull'esito finale, garantendo type-safety e prevenendo stati non previsti dalla norma.

4. Outbound Adapter

Il pacchetto `Outbound Adapter` contiene il `MongoValutazioneAdapter`. Questa classe implementa concretamente il contratto `InterfaceValutazioneRepository`. A differenza del `MongoDispositivoAdapter`, che opera su documenti di primo livello, il `MongoValutazioneAdapter` agisce sull'array `lista_valutazioni` annidato all'interno del sub-documento `lista_asset` del documento `Dispositivo`, come definito nello Schema Dati alla Sezione 3.6.2. Questa scelta è coerente con la struttura di composizione tra le entità e con la strategia di normalizzazione selettiva adottata.

5.5) Importazione ed Esportazione Dispositivi

Per realizzare le funzioni di importazione ed esportazione dei dispositivi tramite file esterno rispettando i principi dell'architettura esagonale si è deciso di modellare il sistema mettendo in evidenza l'ambito di competenza delle varie classi.

5.5.1) Importazione Dispositivo

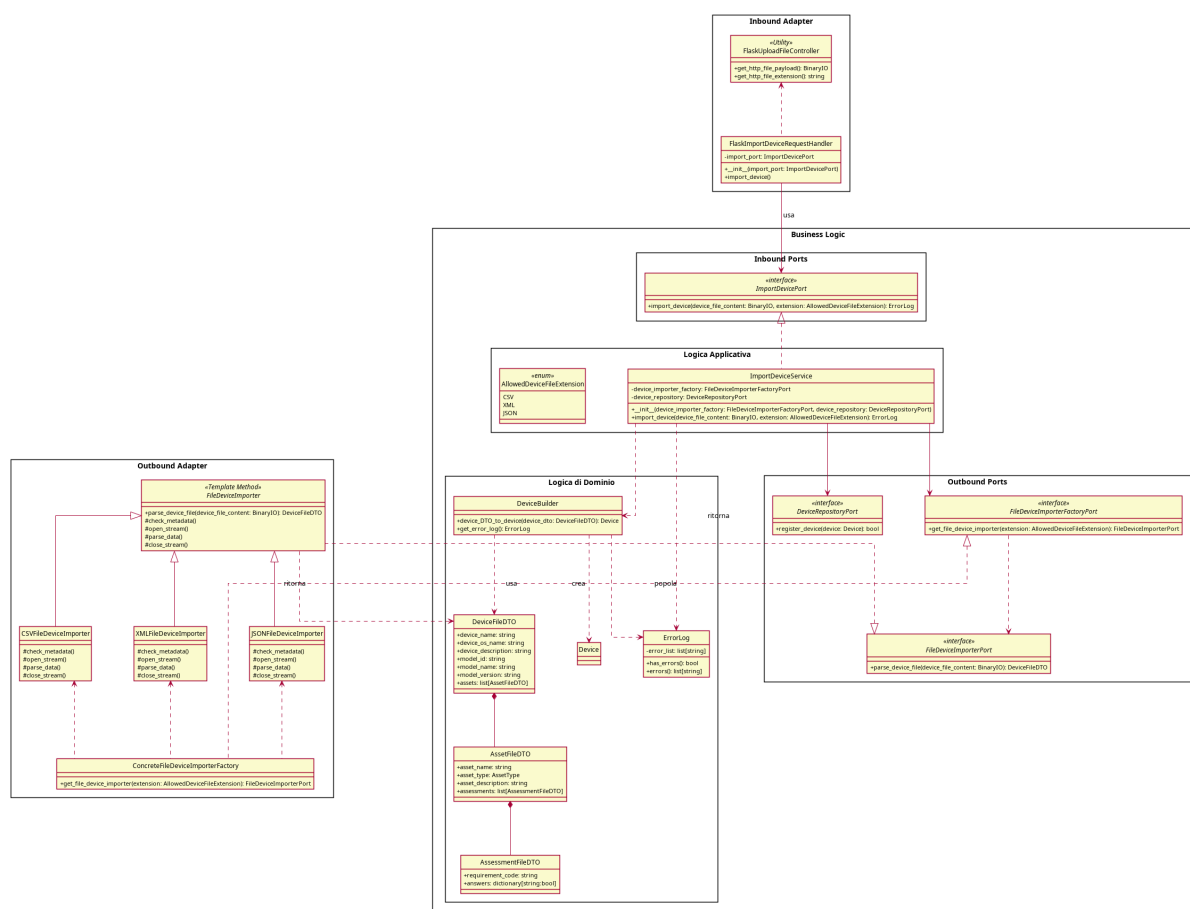


Figura 3: Diagramma delle classi - Importazione Dispositivo

Le classi dell'inbound adapter gestiscono la ricezione dei file dati in ingresso usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta il file come parametro un oggetto di una classe della libreria Python standard

La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: DeviceFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

Per gestire la conversione da DTO a oggetti di dominio si usa una classe utility che converte il DTO in un oggetto di dominio che può essere inoltrato al sistema di permanenza

Vi sono altri servizi definiti esternamente tramite le `outbound ports device file importer` e `device file importer factory`, il `factory` serve a non dare al `service` la responsabilità di creazione dell'importer appropriato al formato di file caricato.

L'importer è implementato tramite un template method le cui implementazioni concrete sono create dal factory

5.5.2) Esportazione dispositivo

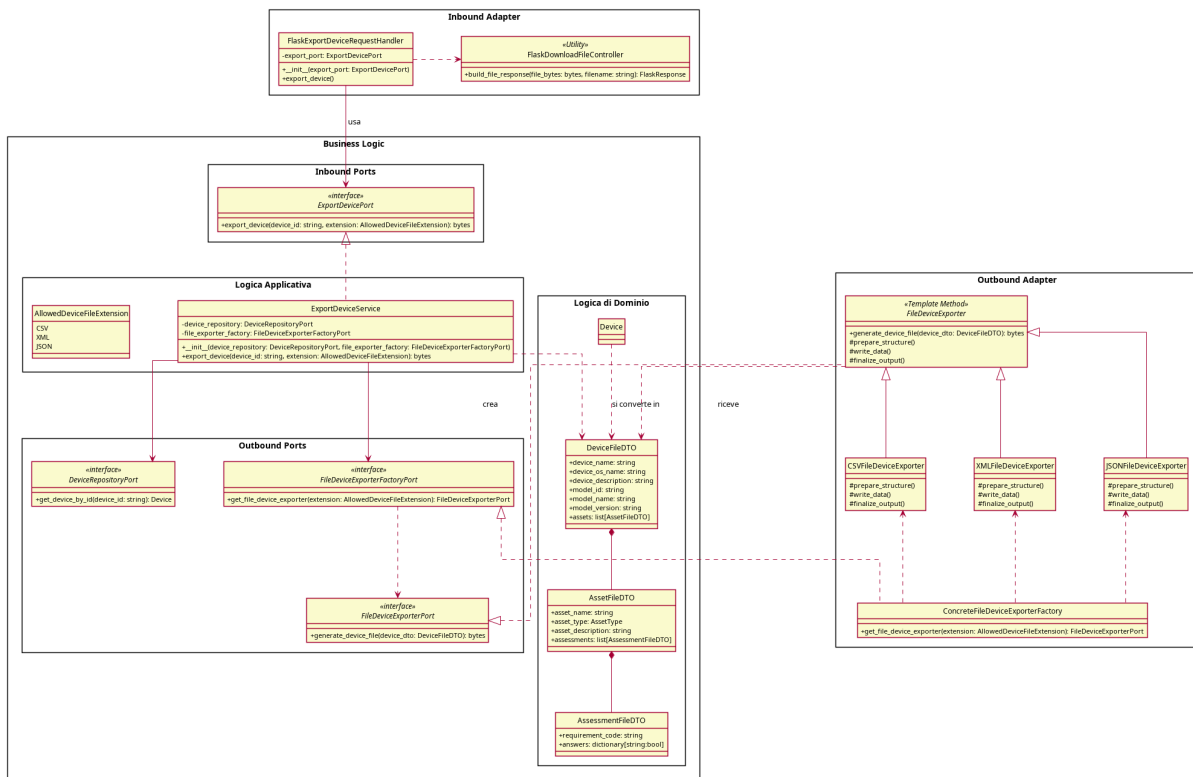


Figura 4: Diagramma delle classi - Esportazione Dispositivo

Le classi dell'inbound adapter gestiscono l'invio del file dati in uscita usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta come parametri un id del dispositivo da esportare e come tipo di ritorno una classe della libreria standard

La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: DeviceFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

Vi sono servizi definiti esternamente tramite le outbound ports device file exporter e device file exporter factory, il factory serve a non dare al service la responsabilità di creazione dell'exporter appropriato al formato di file richiesto.

L'exporter è implementato tramite un template method le cui implementazioni concrete sono create dal factory

5.5.3) Classi Condivise

Essendo importazione ed esportazione 2 processi strettamente collegati, è stato ritenuto accettabile mantenere una classe condivisa di DTO, in quanto i dati necessari sono gli stessi sia durante il processo di importazione sia di esportazione

5.6) Importazione ed Esportazione Modelli

Per realizzare le funzioni di importazione ed esportazione dei modelli tramite file esterno rispettando i principi dell'architettura esagonale si è deciso di modellare il sistema mettendo in evidenza l'ambito di competenza delle varie classi.

5.6.1) Importazione Modello

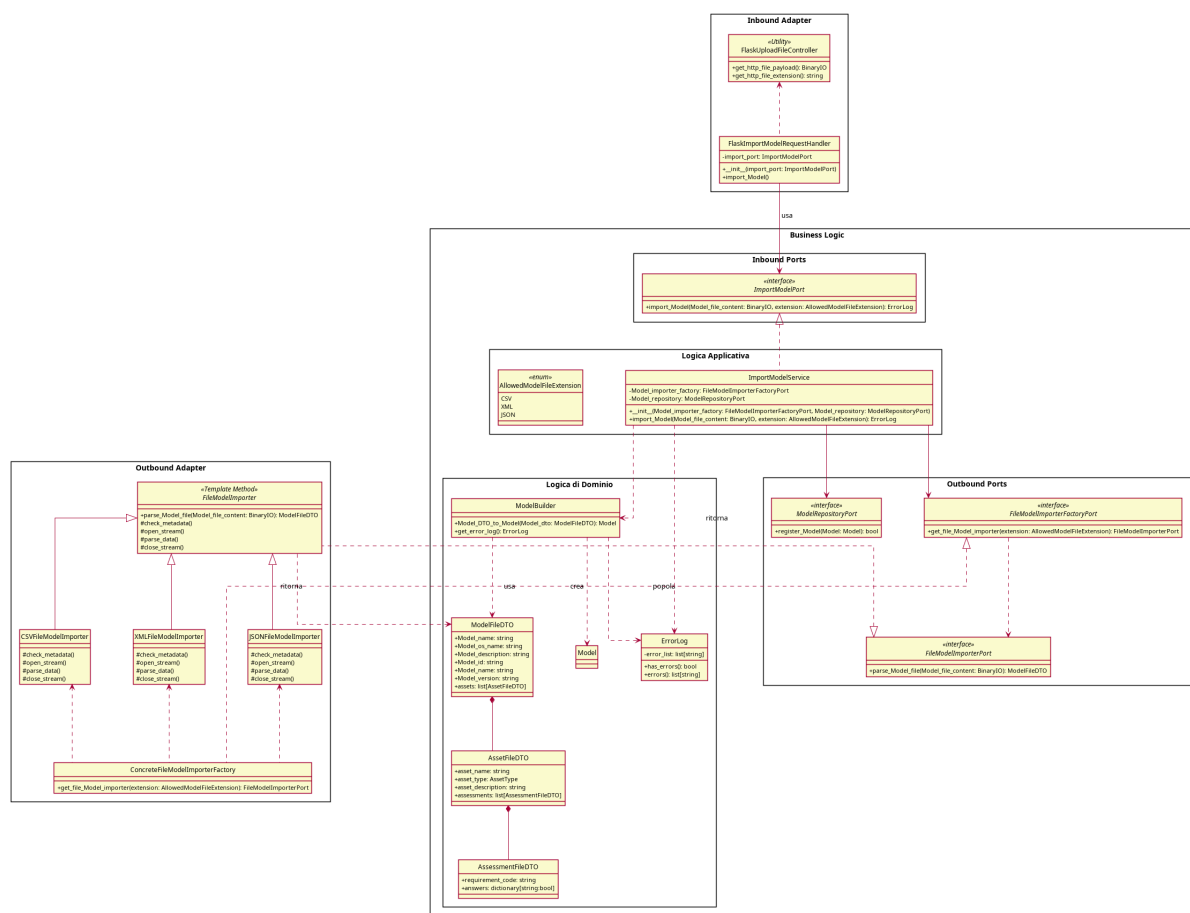


Figura 5: Diagramma delle classi - Importazione Modello

Le classi dell'inbound adapter gestiscono la ricezione dei file dati in ingresso usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta il file come parametro un oggetto di una classe della libreria Python standard

La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: ModelFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

Per gestire la conversione da DTO a oggetti di dominio si usa una classe utility che converte il DTO in un oggetto di dominio che può essere inoltrato al sistema di permanenza

Vi sono altri servizi definiti esternamente tramite le outbound ports model file importer e model file importer factory, il factory serve a non dare al service la responsabilità di creazione dell'importer appropriato al formato di file caricato.

L'importer è implementato tramite un template method le cui implementazioni concrete sono create dal factory

5.6.2) Esportazione modello

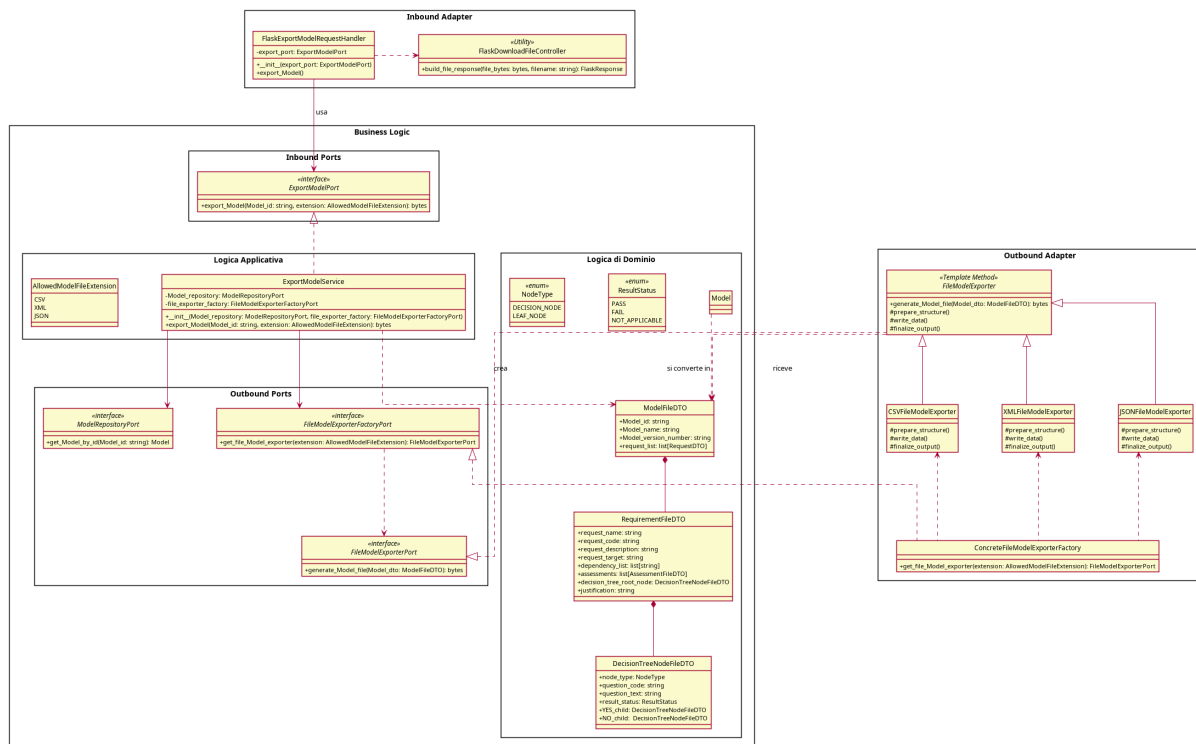


Figura 6: Diagramma delle classi - Esportazione Modello

Le classi dell'inbound adapter gestiscono l'invio del file dati in uscita usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta come parametri un id del modello da esportare e ritorna il file generato come una classe della libreria

La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: ModelFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

Vi sono servizi definiti esternamente tramite le outbound ports model file exporter e model file exporter factory, il factory serve a non dare al service la responsabilità di creazione dell'exporter appropriato al formato di file richiesto.

L'exporter è implementato tramite un template method le cui implementazioni concrete sono create dal factory

5.6.3) Classi Condivise

Essendo importazione ed esportazione 2 processi strettamente collegati, è stato ritenuto accettabile mantenere una classe condivisa di DTO, in quanto i dati necessari sono gli stessi sia durante il processo di importazione sia di esportazione

5.7) Generazione Report

Per realizzare la funzione di generazione del report di valutazione come file scaricabile rispettando i principi dell'architettura esagonale si è deciso di modellare il sistema mettendo in evidenza l'ambito di competenza delle varie classi.

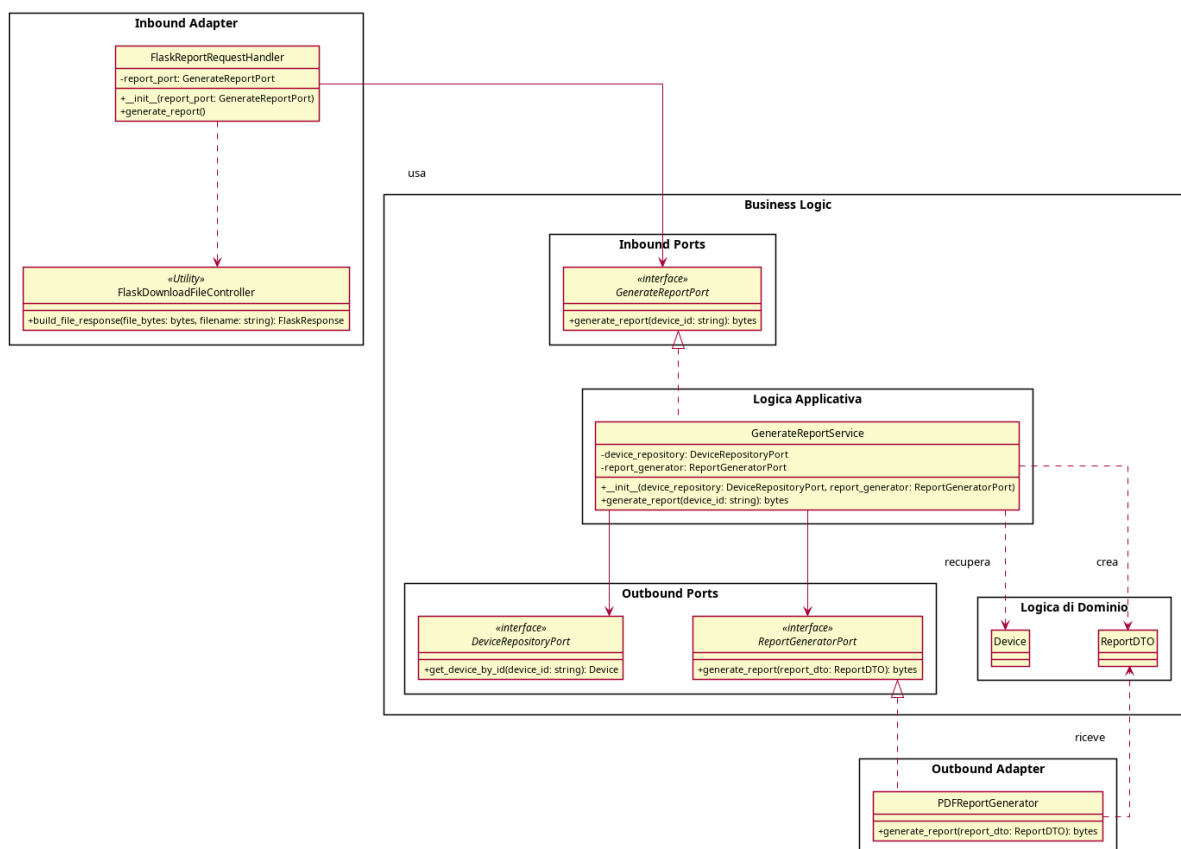


Figura 7: Diagramma delle classi - Importazione Modello

Le classi dell'inbound adapter gestiscono l'invio del file dati in uscita usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta come parametri un id del modello su cui eseguire la valutazione e ritorna il file di report generato come una classe della libreria standard

La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: reportDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

La generazione del file di report è gestita come servizio esterno con una porta dedicata e implementata da una classe esterna, non viene usato un factory in quanto è da supportare la generazione di una sola forma di report.

6) Tracciamento

7) Qualità architettuale

8) Gestione Errori e Logging

9) Sicurezza

10) Performance e Scalabilità